

Analysis of disruptive winds to overwintering monarch butterflies

Kyle Nessen

2025-10-07

Table of contents

Introduction	2
Setup	2
Exploratory Data Analysis	2
Individual Predictor GAM Plots	8
Modeling Strategy	12
Model Building and Selection	12
Model Fitting	14
Model Comparison	15
Best Model Analysis	18
Effect Plots (ggplot2 with Consistent Scaling and Colors)	19
1x4 Grid Layout (Wide Aspect Ratio)	26
Model Diagnostics	27
Residuals vs Fitted Values	27
Q-Q Plot of Residuals	27
Distribution of Residuals	28
Results Summary	28
Wind $\times$ Sun Interaction (Binned, High-Res)	30
Model Selection Table Export	32
Smooth Terms Summary Table Export	35

Publication-Ready Partial Effect Plots (Updated)	36
Publication-Ready Partial Effects (Smart 1x3 without Sun)	46
Model Diagnostics Plots Export	48
Wind Hypothesis Evaluation Plot	49
Data Summary Export	51
Export Summary Report	52

Introduction

This analysis investigates the first hypothesis of my master's thesis: that wind acts as a disruptive force to overwintering monarch butterflies. If true, we predict that monarch abundance at roosts will decrease when exposed to disruptive winds. I use labeled photos from my 2023-2024 dataset to test this hypothesis. I employed GAM (Generalized Additive Models) because they allow for non-linear relationships in fixed effects while maintaining the necessary random effect structure to account for temporal autocorrelation and nested sampling design. All smooth terms used thin plate regression splines (TPRS) as the basis function, which is the default in `mgcv` and provides optimal smoothing for most ecological relationships.

Setup

Load libraries and data:

```
library(tidyverse)
library(mgcv)
library(lubridate)
library(plotly)
library(knitr)
library(DT)
library(here)
library(gratia) # For ggplot2-based GAM plotting
library(patchwork) # For combining ggplot2 plots
# Load the monarch analysis data
monarch_data <- read_csv(here("data", "monarch_analysis_lag30min.csv"))
```

Exploratory Data Analysis

The response variable is the difference in monarch counts between time t and $t-1$ at 30-minute intervals. I applied a cube root transformation to achieve a

more normal distribution. Because the lagged comparisons create overlapping pairs of observations, I include an AR1 autocorrelation structure to account for temporal dependence.

```
knitr::include_graphics("images/clipboard-1435734413.png")
```

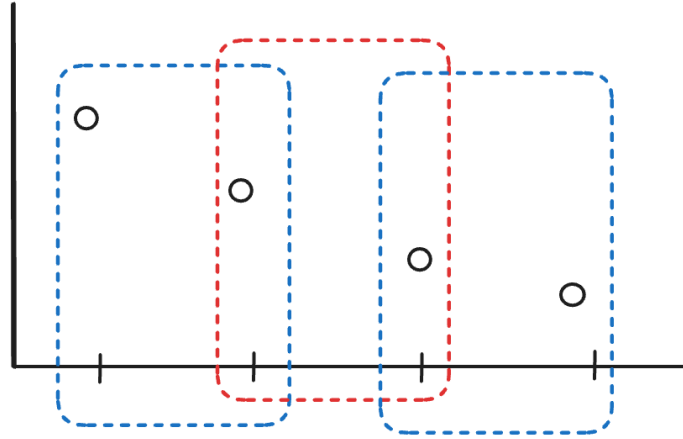


Figure 1: Illustration of temporal dependency in observation pairs. Points represent photos with labeled count data at 30-minute intervals. Blue boxes show non-overlapping pairs of observations. The red box shows an overlapping comparison where one observation is shared between adjacent pairs, creating temporal autocorrelation that is controlled by the AR1 structure.

```
library(gridExtra)

# Compare total butterfly counts with and without SC8
p_all <- ggplot(monarch_data, aes(x = total_butterflies_t_lag)) +
  geom_histogram(bins = 30, fill = "steelblue", alpha = 0.7) +
  labs(
    title = "Total Butterflies - All Data",
    x = "Previous Butterfly Count", y = "Frequency"
  ) +
  theme_minimal()

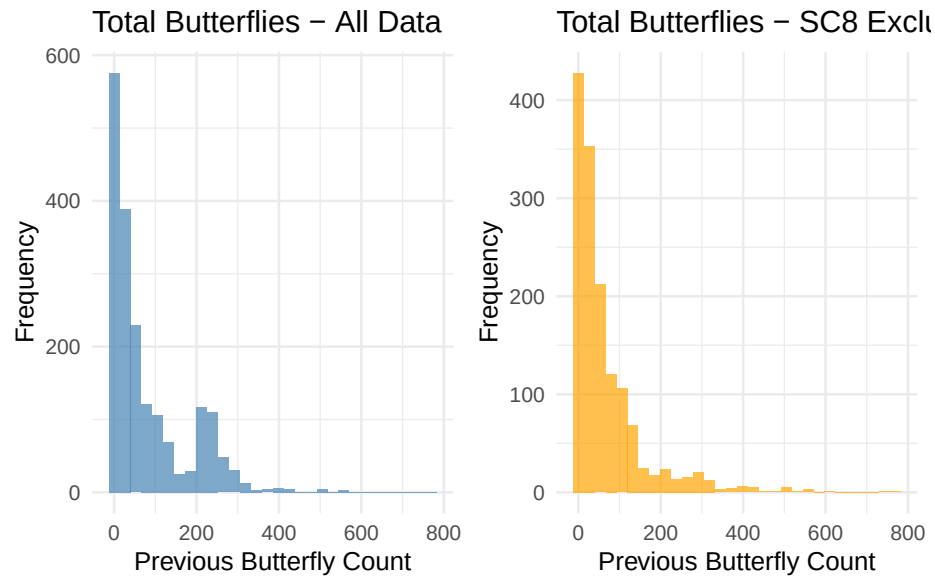
p_no_sc8 <- ggplot(monarch_data %>% filter(deployment_id != "SC8"), aes(x = total_butterflies_t_lag)) +
  geom_histogram(bins = 30, fill = "orange", alpha = 0.7) +
  labs(
    title = "Total Butterflies - SC8 Excluded",
    x = "Previous Butterfly Count", y = "Frequency"
  )
```

```

) +
  theme_minimal()

grid.arrange(p_all, p_no_sc8, ncol = 2)

```



```

library(corrplot)
library(gridExtra)

# Select variables used in the models
model_vars <- monarch_data %>%
  select(
    butterfly_difference_cbrt, total_butterflies_t_lag, max_gust,
    temperature_avg, butterflies_direct_sun_t_lag, time_within_day_t
  )

# Histograms of key variables
p1 <- ggplot(monarch_data, aes(x = butterfly_difference_cbrt)) +
  geom_histogram(bins = 30, fill = "steelblue", alpha = 0.7) +
  labs(
    title = "Response: Butterfly Difference (Cube Root)",
    x = "Butterfly Difference (cbrt)", y = "Frequency"
  )

p2 <- ggplot(monarch_data, aes(x = total_butterflies_t_lag)) +
  geom_histogram(bins = 30, fill = "orange", alpha = 0.7) +
  labs(

```

```

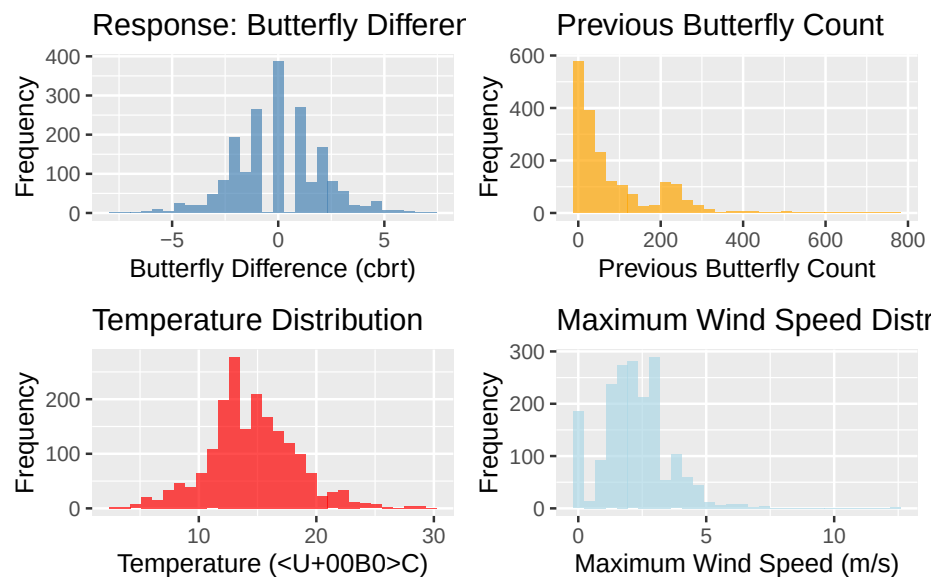
    title = "Previous Butterfly Count",
    x = "Previous Butterfly Count", y = "Frequency"
  )

p3 <- ggplot(monarch_data, aes(x = temperature_avg)) +
  geom_histogram(bins = 30, fill = "red", alpha = 0.7) +
  labs(
    title = "Temperature Distribution",
    x = "Temperature (°C)", y = "Frequency"
  )

p4 <- ggplot(monarch_data, aes(x = max_gust)) +
  geom_histogram(bins = 30, fill = "lightblue", alpha = 0.7) +
  labs(
    title = "Maximum Wind Speed Distribution",
    x = "Maximum Wind Speed (m/s)", y = "Frequency"
  )

grid.arrange(p1, p2, p3, p4, ncol = 2)

```



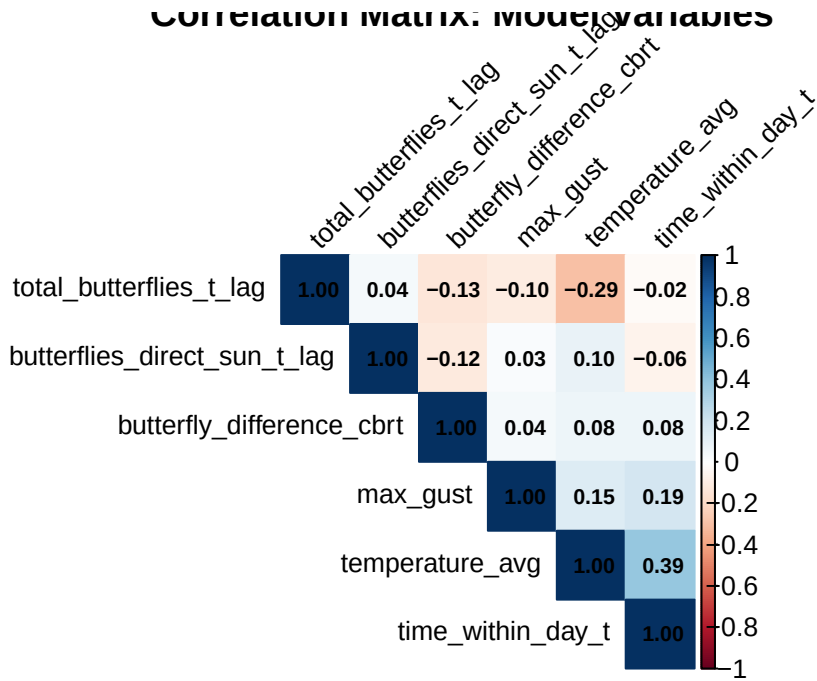
```

# Correlation matrix for model variables
cor_matrix <- cor(model_vars, use = "complete.obs")

# Create correlation plot
corrplot(cor_matrix,
  method = "color",

```

```
type = "upper",
order = "hclust",
tl.cex = 0.8,
tl.col = "black",
tl.srt = 45,
addCoef.col = "black",
number.cex = 0.7,
title = "Correlation Matrix: Model Variables"
)
```



```
# Print correlation table
kable(round(cor_matrix, 3),
caption = "Correlation Matrix for Model Variables"
)
```

Table 1: Correlation Matrix for Model Variables

	butterfly_difference_cbrt	total_butterflies_t_lag	max_gust	temperature_avg	butterflies_direct_sun_t_lag	time_within_day_t
butterfly_difference_cbrt	1.000	-0.131	0.040	0.079	-0.116	0.077
total_butterflies_t_lag		1.000	-	-0.291	0.041	-0.023
max_gust			1.000	0.145	0.027	0.185
temperature_avg				1.000		
butterflies_direct_sun_t_lag					1.000	
time_within_day_t						1.000

	butterfly_difference_cb	total_butterflies_t	temperature_avg	butterflies_direct_sun_t	time_within_day_t
temperature_avg	0.079	-0.291	0.145	1.000	0.099
butterflies_direct_sun_t	0.041	0.027	0.099	1.000	-0.064
time_within_day_t	0.077	-0.023	0.185	0.386	-0.064

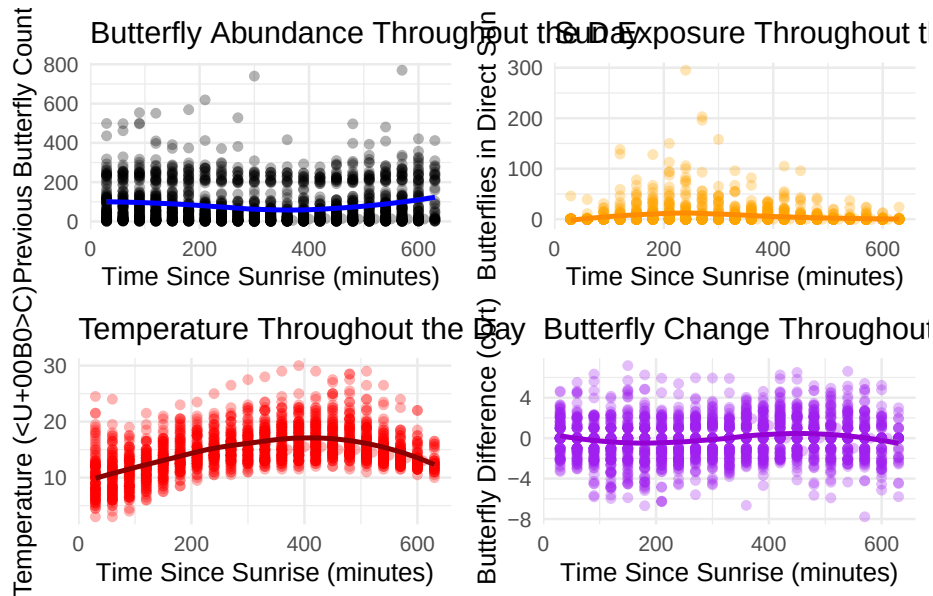
```
# Butterfly activity by time of day
p1 <- ggplot(monarch_data, aes(x = time_within_day_t, y = total_butterflies_t_lag)) +
  geom_point(alpha = 0.3) +
  geom_smooth(method = "loess", se = TRUE, color = "blue") +
  labs(
    title = "Butterfly Abundance Throughout the Day",
    x = "Time Since Sunrise (minutes)", y = "Previous Butterfly Count"
  ) +
  theme_minimal()

# Sun exposure patterns by time
p2 <- ggplot(monarch_data, aes(x = time_within_day_t, y = butterflies_direct_sun_t_lag)) +
  geom_point(alpha = 0.3, color = "orange") +
  geom_smooth(method = "loess", se = TRUE, color = "darkorange") +
  labs(
    title = "Sun Exposure Throughout the Day",
    x = "Time Since Sunrise (minutes)", y = "Butterflies in Direct Sun"
  ) +
  theme_minimal()

# Temperature patterns by time
p3 <- ggplot(monarch_data, aes(x = time_within_day_t, y = temperature_avg)) +
  geom_point(alpha = 0.3, color = "red") +
  geom_smooth(method = "loess", se = TRUE, color = "darkred") +
  labs(
    title = "Temperature Throughout the Day",
    x = "Time Since Sunrise (minutes)", y = "Temperature (°C)"
  ) +
  theme_minimal()

# Response variable by time
p4 <- ggplot(monarch_data, aes(x = time_within_day_t, y = butterfly_difference_cb)) +
  geom_point(alpha = 0.3, color = "purple") +
  geom_smooth(method = "loess", se = TRUE, color = "darkviolet") +
  labs(
    title = "Butterfly Change Throughout the Day",
    x = "Time Since Sunrise (minutes)", y = "Butterfly Difference (cbt)"
  ) +
  theme_minimal()
```

```
grid.arrange(p1, p2, p3, p4, ncol = 2)
```



Individual Predictor GAM Plots

Exploring the relationship between each model predictor and butterfly_difference_cbirt with GAM smoothed lines to understand the shape of relationships before modeling.

```
library(mgcv)

# Maximum Wind Speed vs Butterfly Difference
p1 <- ggplot(monarch_data, aes(x = max_gust, y = butterfly_difference_cbirt)) +
  geom_point(alpha = 0.3, color = "steelblue") +
  geom_smooth(method = "gam", se = TRUE, color = "darkblue") +
  labs(
    title = "Maximum Wind Speed vs Butterfly Change",
    x = "Maximum Wind Speed (m/s)",
    y = "Butterfly Difference (cbirt)"
  ) +
  theme_minimal()

# Temperature vs Butterfly Difference
p2 <- ggplot(monarch_data, aes(x = temperature_avg, y = butterfly_difference_cbirt)) +
  geom_point(alpha = 0.3, color = "red") +
```

```

    geom_smooth(method = "gam", se = TRUE, color = "darkred") +
    labs(
      title = "Temperature vs Butterfly Change",
      x = "Temperature (°C)",
      y = "Butterfly Difference (cbrt)"
    ) +
    theme_minimal()

# Butterflies in Direct Sun vs Butterfly Difference
p3 <- ggplot(monarch_data, aes(x = butterflies_direct_sun_t_lag, y = butterfly_difference_cb)) +
  geom_point(alpha = 0.3, color = "orange") +
  geom_smooth(method = "gam", se = TRUE, color = "darkorange") +
  labs(
    title = "Direct Sun Exposure vs Butterfly Change",
    x = "Butterflies in Direct Sun",
    y = "Butterfly Difference (cbrt)"
  ) +
  theme_minimal()

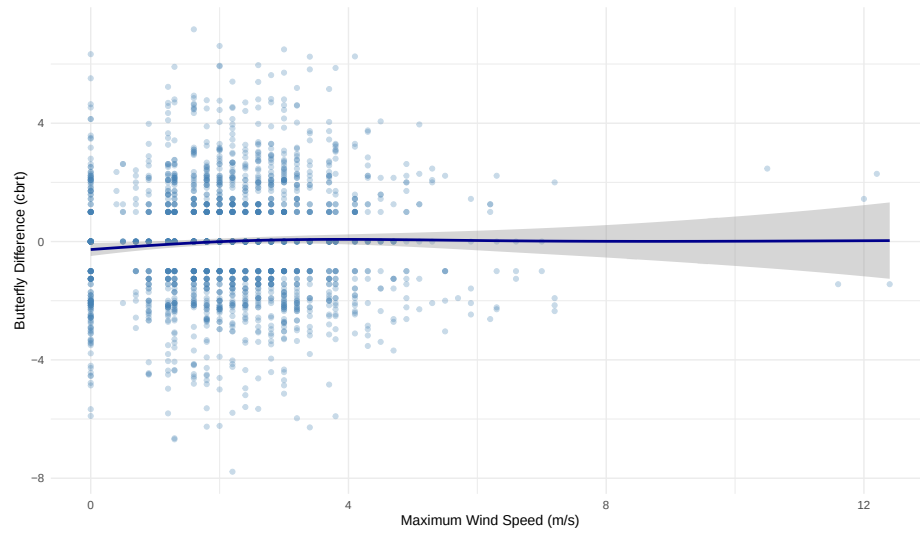
# Time of Day vs Butterfly Difference
p4 <- ggplot(monarch_data, aes(x = time_within_day_t, y = butterfly_difference_cb)) +
  geom_point(alpha = 0.3, color = "purple") +
  geom_smooth(method = "gam", se = TRUE, color = "darkviolet") +
  labs(
    title = "Time Since Sunrise vs Butterfly Change",
    x = "Time Since Sunrise (minutes)",
    y = "Butterfly Difference (cbrt)"
  ) +
  theme_minimal()

# Total Butterflies (previous) vs Butterfly Difference
p5 <- ggplot(monarch_data, aes(x = total_butterflies_t_lag, y = butterfly_difference_cb)) +
  geom_point(alpha = 0.3, color = "darkgreen") +
  geom_smooth(method = "gam", se = TRUE, color = "forestgreen") +
  labs(
    title = "Previous Butterfly Count vs Change",
    x = "Previous Butterfly Count",
    y = "Butterfly Difference (cbrt)"
  ) +
  theme_minimal()

# Display plots in grid
p1

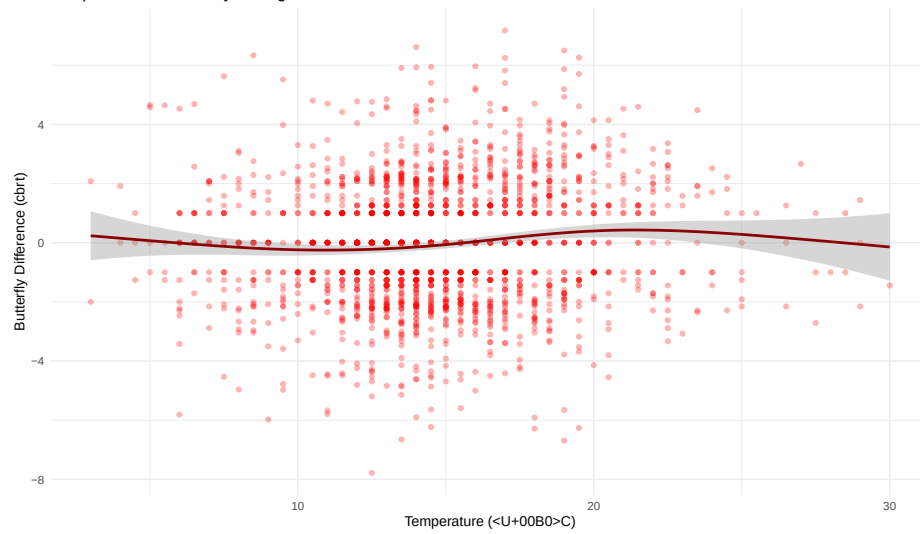
```

Maximum Wind Speed vs Butterfly Change



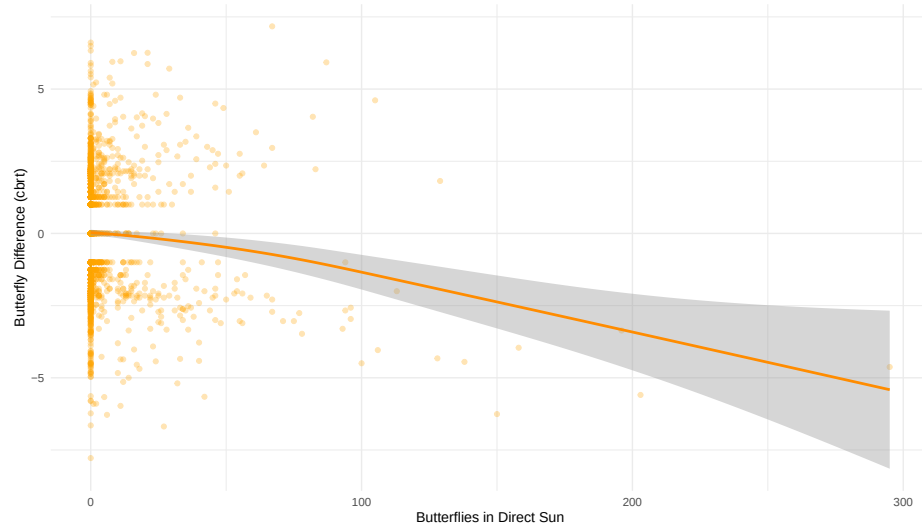
p2

Temperature vs Butterfly Change



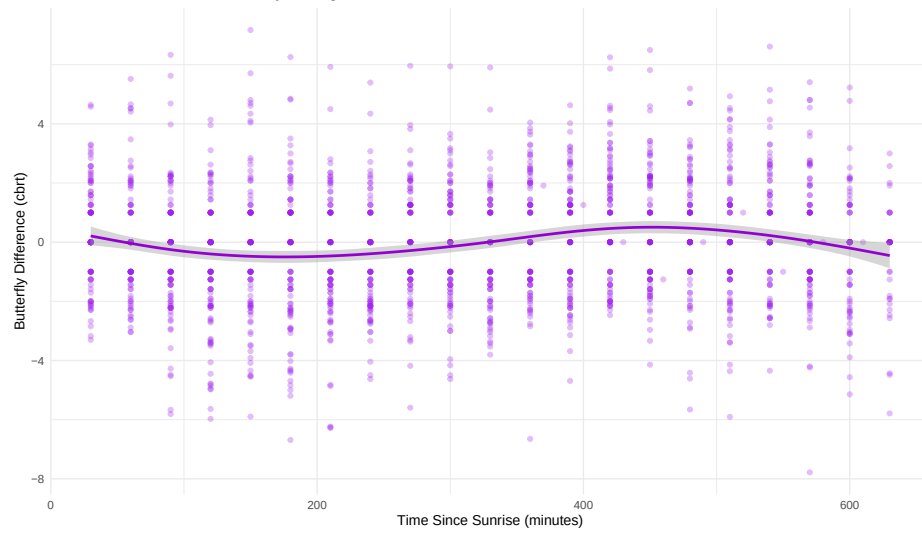
p3

Direct Sun Exposure vs Butterfly Change

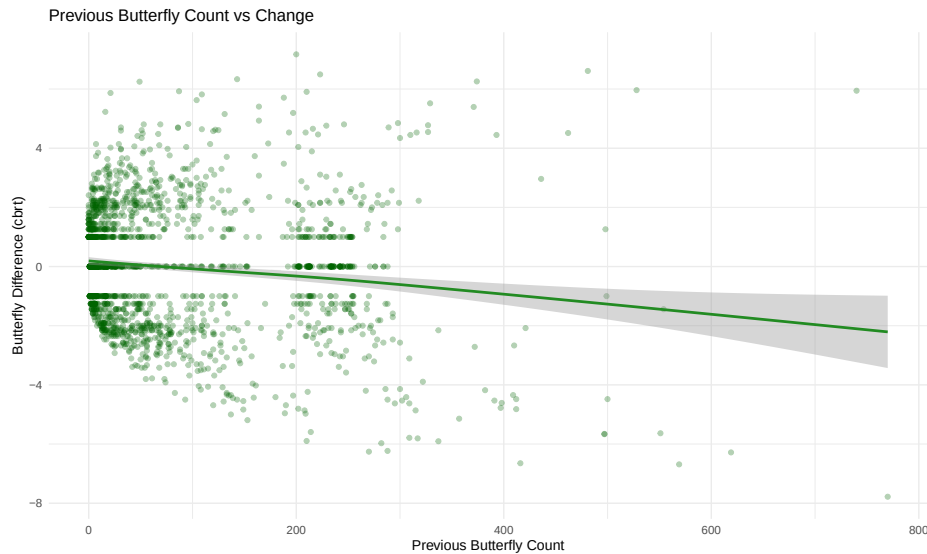


p4

Time Since Sunrise vs Butterfly Change



p5



Modeling Strategy

Our modeling approach used a comprehensive AIC-based comparison to evaluate all possible combinations of three key environmental predictors: wind speed (`max_gust`), temperature (`temperature_avg`), and solar exposure (`butterflies_direct_sun_t_lag`). We tested two fundamental modeling frameworks: models that include `total_butterflies_t_lag` as a control variable (testing effects on relative/proportional change) and models that exclude it (testing effects on absolute change). Within each framework, we systematically evaluated linear main effects, two-way and three-way interactions, and non-linear relationships using smooth terms. We also incorporated time-of-day effects to capture diurnal patterns. This resulted in 47 candidate models that comprehensively explore the parameter space while maintaining proper mixed-effects structure with random effects for deployment, observer, and day, plus AR1 correlation for within-day autocorrelation.

Model Building and Selection

Please expand the code block to see the full list of models tested.

```
library(nlme)

# Define the random effects structure and correlation
random_structure <- list(deployment_id = ~1, Observer = ~1, deployment_day = ~1)
correlation_structure <- corAR1(form = ~ observation_order_within_day_t | deployment_day)
```

```

# Model specifications for AIC comparison
model_specs <- list(
  # Null model
  "M1" = "butterfly_difference_cbrt ~ total_butterflies_t_lag",

  # Single variable models
  "M2" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust",
  "M3" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + temperature_avg",
  "M4" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + butterflies_direct_sun_t_lag",

  # Two-variable combinations
  "M5" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust + temperature_avg",
  "M6" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust + butterflies_direct_sun_t_lag",
  "M7" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + temperature_avg + butterflies_direct_sun_t_lag",

  # Three-variable model (main effects only)
  "M8" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust + temperature_avg + butterflies_direct_sun_t_lag",

  # Two-way interactions
  "M9" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust * temperature_avg",
  "M10" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust * butterflies_direct_sun_t_lag",
  "M11" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + temperature_avg * butterflies_direct_sun_t_lag",

  # Two-way interactions with third variable as main effect
  "M12" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust * temperature_avg + butterflies_direct_sun_t_lag",
  "M13" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust * butterflies_direct_sun_t_lag + temperature_avg",
  "M14" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + temperature_avg * butterflies_direct_sun_t_lag + max_gust",

  # All two-way interactions
  "M15" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust * temperature_avg + butterflies_direct_sun_t_lag",

  # Three-way interaction
  "M16" = "butterfly_difference_cbrt ~ total_butterflies_t_lag + max_gust * temperature_avg + butterflies_direct_sun_t_lag",

  # Smooth terms models (with lag term)
  "M17" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(temperature_avg) + s(butterflies_direct_sun_t_lag)",
  "M18" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + temperature_avg + s(butterflies_direct_sun_t_lag)",
  "M19" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(max_gust) + temperature_avg + s(butterflies_direct_sun_t_lag)",
  "M20" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(temperature_avg) + s(butterflies_direct_sun_t_lag)",
  "M21" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(max_gust) + s(temperature_avg) + s(butterflies_direct_sun_t_lag)",
  "M22" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + temperature_avg + s(butterflies_direct_sun_t_lag)",
  "M23" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(temperature_avg) + s(butterflies_direct_sun_t_lag)",
  "M24" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(max_gust) + s(temperature_avg) + s(butterflies_direct_sun_t_lag)",

  # Models WITHOUT lag term - testing environmental effects on absolute change

```

```

"M25" = "butterfly_difference_cbrt ~ 1",
"M26" = "butterfly_difference_cbrt ~ max_gust",
"M27" = "butterfly_difference_cbrt ~ temperature_avg",
"M28" = "butterfly_difference_cbrt ~ butterflies_direct_sun_t_lag",
"M29" = "butterfly_difference_cbrt ~ max_gust + temperature_avg",
"M30" = "butterfly_difference_cbrt ~ max_gust + butterflies_direct_sun_t_lag",
"M31" = "butterfly_difference_cbrt ~ temperature_avg + butterflies_direct_sun_t_lag",
"M32" = "butterfly_difference_cbrt ~ max_gust + temperature_avg + butterflies_direct_sun_t_lag",
"M33" = "butterfly_difference_cbrt ~ max_gust * temperature_avg",
"M34" = "butterfly_difference_cbrt ~ max_gust * butterflies_direct_sun_t_lag",
"M35" = "butterfly_difference_cbrt ~ temperature_avg * butterflies_direct_sun_t_lag",
"M36" = "butterfly_difference_cbrt ~ max_gust * temperature_avg + butterflies_direct_sun_t_lag",
"M37" = "butterfly_difference_cbrt ~ max_gust * butterflies_direct_sun_t_lag + temperature_avg",
"M38" = "butterfly_difference_cbrt ~ temperature_avg * butterflies_direct_sun_t_lag + max_gust",
"M39" = "butterfly_difference_cbrt ~ max_gust * temperature_avg + max_gust * butterflies_direct_sun_t_lag",
"M40" = "butterfly_difference_cbrt ~ max_gust * temperature_avg * butterflies_direct_sun_t_lag",

# Smooth terms models WITHOUT lag term
"M41" = "butterfly_difference_cbrt ~ s(temperature_avg) + s(butterflies_direct_sun_t_lag)",
"M42" = "butterfly_difference_cbrt ~ temperature_avg + s(butterflies_direct_sun_t_lag)",
"M43" = "butterfly_difference_cbrt ~ s(max_gust) + temperature_avg + s(butterflies_direct_sun_t_lag)",
"M44" = "butterfly_difference_cbrt ~ s(temperature_avg) + s(butterflies_direct_sun_t_lag) + s(max_gust)",
"M45" = "butterfly_difference_cbrt ~ s(max_gust) + s(temperature_avg) + s(butterflies_direct_sun_t_lag)",
"M46" = "butterfly_difference_cbrt ~ temperature_avg + s(butterflies_direct_sun_t_lag) + s(max_gust)",
"M47" = "butterfly_difference_cbrt ~ s(temperature_avg) + s(butterflies_direct_sun_t_lag) + s(max_gust)",
"M48" = "butterfly_difference_cbrt ~ s(max_gust) + s(temperature_avg) + s(butterflies_direct_sun_t_lag)",

# NEW: Tensor-product interaction candidates (wind × sun) for parity with daily analysis
# With lag term and diurnal controls
"M49" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + ti(max_gust, butterflies_direct_sun_t_lag)",
"M50" = "butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(temperature_avg) + s(max_gust)",
# Without lag term (absolute change)
"M51" = "butterfly_difference_cbrt ~ ti(max_gust, butterflies_direct_sun_t_lag)",
"M52" = "butterfly_difference_cbrt ~ s(temperature_avg) + s(time_within_day_t) + ti(max_gust, butterflies_direct_sun_t_lag)",
)

cat("Total models to fit:", length(model_specs), "\n")

```

Total models to fit: 52

Model Fitting

```

# Function to safely fit models
fit_model_safely <- function(formula_str, data) {
  tryCatch({
    formula_obj <- as.formula(formula_str)
    gamm(formula_obj,
      data = data,
      random = random_structure,
      correlation = correlation_structure,
      method = "REML"
    )
  },
  error = function(e) {
    message("Failed to fit model: ", formula_str)
    message("Error: ", e$message)
    return(NULL)
  }
)
}

# Fit all models
cat("Fitting models...\n")

```

Fitting models...

```

fitted_models <- map(model_specs, ~ fit_model_safely(.x, model_data))

# Remove failed models
successful_models <- fitted_models[!map_lgl(fitted_models, is.null)]
cat("Successfully fitted", length(successful_models), "out of", length(model_specs), "models")

```

Successfully fitted 52 out of 52 models

Model Comparison

```

# Extract AIC values
aic_results <- map_dfr(names(successful_models), function(model_name) {
  model <- successful_models[[model_name]]
  data.frame(
    Model = model_name,
    Formula = model_specs[[model_name]],
    AIC = AIC(model$lme),
  )
})

```

```

      LogLik = logLik(model$lme)[1],
      df = attr(logLik(model$lme), "df")
    )
  }) %>%
  arrange(AIC) %>%
  mutate(
    Delta_AIC = AIC - min(AIC),
    AIC_weight = exp(-0.5 * Delta_AIC) / sum(exp(-0.5 * Delta_AIC))
  )

# Display results
aic_results %>%
  select(Model, AIC, Delta_AIC, AIC_weight, df) %>%
  kable(digits = 3, caption = "Model comparison by AIC")

```

Table 2: Model comparison by AIC

Model	AIC	Delta_AIC	AIC_weight	df
M50	8078.029	0.000	0.856	15
M23	8081.848	3.820	0.127	14
M22	8086.644	8.615	0.012	13
M24	8088.049	10.020	0.006	16
M52	8096.722	18.693	0.000	13
M47	8101.296	23.268	0.000	12
M17	8105.876	27.847	0.000	12
M20	8105.876	27.847	0.000	12
M48	8107.724	29.695	0.000	14
M46	8108.295	30.267	0.000	11
M21	8109.249	31.221	0.000	14
M18	8114.431	36.402	0.000	11
M49	8114.690	36.661	0.000	11
M19	8119.075	41.047	0.000	13
M41	8126.061	48.032	0.000	10
M44	8126.061	48.032	0.000	10
M45	8127.871	49.843	0.000	12
M7	8130.775	52.747	0.000	9
M4	8131.696	53.667	0.000	8
M16	8132.647	54.618	0.000	14
M6	8134.945	56.917	0.000	9
M12	8135.392	57.363	0.000	11
M8	8136.217	58.188	0.000	10
M40	8137.407	59.378	0.000	13
M42	8139.237	61.208	0.000	9
M10	8139.410	61.381	0.000	10

Model	AIC	Delta_AIC	AIC_weight	df
M13	8140.795	62.766	0.000	11
M36	8141.931	63.902	0.000	10
M43	8142.038	64.009	0.000	11
M31	8142.927	64.898	0.000	8
M11	8144.554	66.525	0.000	10
M51	8145.782	67.753	0.000	9
M32	8146.374	68.345	0.000	9
M37	8148.813	70.784	0.000	10
M14	8150.004	71.975	0.000	11
M1	8153.582	75.553	0.000	7
M30	8154.129	76.100	0.000	8
M28	8155.073	77.044	0.000	7
M15	8155.075	77.047	0.000	13
M35	8156.678	78.649	0.000	9
M34	8156.943	78.914	0.000	9
M3	8157.623	79.595	0.000	8
M2	8157.885	79.856	0.000	8
M39	8160.095	82.066	0.000	12
M38	8160.174	82.145	0.000	10
M9	8162.939	84.910	0.000	10
M5	8163.059	85.030	0.000	9
M27	8170.575	92.547	0.000	7
M33	8171.945	93.916	0.000	9
M29	8175.113	97.084	0.000	8
M25	8177.191	99.162	0.000	6
M26	8178.495	100.466	0.000	7

```
# Show top 5 models
cat("\nTop 5 models by AIC:\n")
```

Top 5 models by AIC:

```
head(aic_results, 5) %>%
  select(Model, Formula, AIC, Delta_AIC) %>%
  kable(digits = 3)
```

Model	Formula	AIC	Delta_AIC
M50	butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(temperature_avg) + s(time_within_day_t) + ti(max_gust, butterflies_direct_sun_t_lag)	8078.00	0.000
M23	butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(temperature_avg) + s(butterflies_direct_sun_t_lag) + s(time_within_day_t)	8081.83	3.820
M22	butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + temperature_avg + s(butterflies_direct_sun_t_lag) + s(time_within_day_t)	8086.64	8.615
M24	butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(max_gust) + s(temperature_avg) + s(butterflies_direct_sun_t_lag) + s(time_within_day_t)	8088.04	10.020
M52	butterfly_difference_cbrt ~ s(temperature_avg) + s(time_within_day_t) + ti(max_gust, butterflies_direct_sun_t_lag)	8096.72	8.693

Best Model Analysis

```
# Get the best model
best_model_name <- aic_results$Model[1]
best_model <- successful_models[[best_model_name]]

cat("Best model:", best_model_name, "\n")
```

Best model: M50

```
cat("Formula:", aic_results$Formula[1], "\n\n")
```

Formula: butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(temperature_avg) + s(time_within_day_t) + ti(max_gust, butterflies_direct_sun_t_lag)

```
# Model summary
summary(best_model$gam)
```

Family: gaussian
Link function: identity

Formula:
butterfly_difference_cbrt ~ s(total_butterflies_t_lag) + s(temperature_avg) + s(time_within_day_t) + ti(max_gust, butterflies_direct_sun_t_lag)

Parametric coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	0.1595	0.3825	0.417	0.677

Approximate significance of smooth terms:

	edf	Ref.df	F	p-value
s(total_butterflies_t_lag)	2.409	2.409	12.500	9.98e-07 ***
s(temperature_avg)	3.674	3.674	3.193	0.0571 .
s(time_within_day_t)	4.870	4.870	9.845	< 2e-16 ***
ti(max_gust,butterflies_direct_sun_t_lag)	7.348	7.348	4.666	5.55e-05 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.0636

Scale est. = 4.034 n = 1894

```
# Record available smooth terms and whether expected smooths are present
available_smooths <- rownames(summary(best_model$gam)$s.table)
expected_smooths <- c(
  "s(total_butterflies_t_lag)",
  "s(temperature_avg)",
  "s(butterflies_direct_sun_t_lag)",
  "s(time_within_day_t)"
)
has_expected <- all(expected_smooths %in% available_smooths)
cat("Has expected smooths:", has_expected, "\n")
```

Has expected smooths: FALSE

Effect Plots (ggplot2 with Consistent Scaling and Colors)

```
# Define custom theme with grid lines and darker zero line
custom_theme <- theme_minimal(base_size = 12) +
  theme(
    panel.grid.major = element_line(color = "gray90", size = 0.5),
    panel.grid.minor = element_line(color = "gray95", size = 0.3),
    axis.text = element_text(color = "black", size = 11),
    axis.title = element_text(color = "black", size = 12, face = "bold"),
    plot.title = element_text(color = "black", size = 14, face = "bold", hjust = 0.5),
    panel.border = element_rect(color = "black", fill = NA, size = 0.5),
    plot.margin = margin(10, 10, 10, 10)
  )
```

```

# Function to add darker zero line BEHIND the plot elements
add_zero_line <- function(plot) {
  # Create zero line layer - darker than grid lines but neutral and subtle
  zero_line_layer <- geom_hline(yintercept = 0, color = "gray70", size = 0.8, alpha = 1)

  # Insert zero line as the first layer (background)
  plot$layers <- c(list(zero_line_layer), plot$layers)

  return(plot)
}

# Load required libraries
library(gratia)
library(patchwork)

if (isTRUE(has_expected)) {
  # First, create all individual plots to extract their y-axis ranges
  temp_plots <- list(
    draw(best_model$gam, select = "s(total_butterflies_t_lag)", rug = FALSE),
    draw(best_model$gam, select = "s(temperature_avg)", rug = FALSE),
    draw(best_model$gam, select = "s(butterflies_direct_sun_t_lag)", rug = FALSE),
    draw(best_model$gam, select = "s(time_within_day_t)", rug = FALSE)
  )

  # Extract y-axis ranges from the temporary plots
  y_ranges <- sapply(temp_plots, function(p) {
    built_plot <- ggplot_build(p)
    built_plot$layout$panel_scales_y[[1]]$range$range
  })

  # Use calculated min and clipped max with increments of 1
  y_min <- floor(min(y_ranges))
  y_max <- 1.5 # Clip top to remove empty space
  y_limits <- c(y_min, y_max)
  y_breaks <- seq(y_min, y_max, by = 1)

  cat("Calculated y-axis limits:", y_limits, "\n")
  cat("Y-axis breaks:", y_breaks, "\n")
} else {
  # Fallback axis settings when expected smooths are not present
  y_limits <- c(-2, 1.5)
  y_breaks <- seq(-2, 1.5, by = 1)
  cat("Calculated y-axis limits (default):", y_limits, "\n")
}

```

Calculated y-axis limits (default): -2 1.5

```
# Skip detailed effect plots if expected smooths are missing
if (!isTRUE(has_expected)) {
  cat("Skipping detailed effect plots: expected smooths not present in best model\n")
} else {
  # Define custom colors for each variable (from your color palette)
  # Order: Butterflies in Direct Sun, Previous Butterfly Count, Time Since Sunrise, Temperature
  plot_colors <- c(
    "#c8743b", # Butterflies in Direct Sun (orange/brown)
    "#9673c5", # Previous Butterfly Count (purple)
    "#79a44c", # Time Since Sunrise (green)
    "#b86e7e"  # Temperature (mauve/pink)
  )

  # Function to create 10% lighter colors for shaded areas
  lighten_color <- function(hex_color, amount = 0.1) {
    # Convert hex to RGB
    rgb_vals <- col2rgb(hex_color)
    # Lighten by moving toward white (255)
    lightened <- rgb_vals + (255 - rgb_vals) * amount
    # Convert back to hex
    rgb(lightened[1], lightened[2], lightened[3], maxColorValue = 255)
  }

  # Create lighter versions for shaded areas
  plot_colors_light <- sapply(plot_colors, lighten_color)

  # Create individual plots with consistent y-axis and unique colors
  # Based on your rearrangement: Butterflies in Direct Sun, Previous Butterfly Count, Time Since Sunrise, Temperature
  # Only the first plot (Butterflies in Direct Sun) will have y-axis label to reduce redundancy

  p1 <- draw(best_model$gam, select = "s(butterflies_direct_sun_t_lag)",
    rug = FALSE, residuals = FALSE) +
    labs(
      x = "Butterflies in direct sun",
      y = "Partial effect",
      title = NULL # Remove title
    ) +
    scale_y_continuous(limits = y_limits, breaks = y_breaks) +
    custom_theme +
    theme(
      legend.position = "none",
      plot.background = element_rect(fill = "white", color = NA),
      panel.background = element_rect(fill = "white", color = NA),

```

```

    plot.caption = element_blank() # Remove basis caption
  )

# Add darker zero line BEFORE applying color changes
p1 <- add_zero_line(p1)

# Override the default colors using a more direct approach (skip zero line)
# Modify all layers to use custom colors
for (i in seq_along(p1$layers)) {
  # Skip the zero line (GeomHline) to preserve its neutral gray color
  if (!is.null(p1$layers[[i]]$geom) && inherits(p1$layers[[i]]$geom, "GeomHline")) {
    next
  }

  if ("colour" %in% names(p1$layers[[i]]$aes_params)) {
    p1$layers[[i]]$aes_params$colour <- plot_colors[1]
  }
  if ("fill" %in% names(p1$layers[[i]]$aes_params)) {
    p1$layers[[i]]$aes_params$fill <- plot_colors_light[1]
  }
  # Also try to override any default aesthetics
  if (!is.null(p1$layers[[i]]$geom)) {
    if (inherits(p1$layers[[i]]$geom, "GeomLine")) {
      p1$layers[[i]]$aes_params$colour <- plot_colors[1]
    }
    if (inherits(p1$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      p1$layers[[i]]$aes_params$fill <- plot_colors_light[1]
    }
  }
}

p2 <- draw(best_model$gam, select = "s(total_butterflies_t_lag)",
           rug = FALSE, residuals = FALSE) +
  labs(
    x = "Previous butterfly count",
    y = "", # Remove y-axis label
    title = NULL # Remove title
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  custom_theme +
  theme(
    legend.position = "none",
    plot.background = element_rect(fill = "white", color = NA),
    panel.background = element_rect(fill = "white", color = NA),
    axis.text.y = element_blank(), # Remove y-axis text

```

```

    axis.ticks.y = element_blank(), # Remove y-axis ticks
    plot.caption = element_blank() # Remove basis caption
  )

# Add darker zero line BEFORE applying color changes
p2 <- add_zero_line(p2)

for (i in seq_along(p2$layers)) {
  # Skip the zero line (GeomHline) to preserve its neutral gray color
  if (!is.null(p2$layers[[i]]$geom) && inherits(p2$layers[[i]]$geom, "GeomHline")) {
    next
  }

  if ("colour" %in% names(p2$layers[[i]]$aes_params)) {
    p2$layers[[i]]$aes_params$colour <- plot_colors[2]
  }
  if ("fill" %in% names(p2$layers[[i]]$aes_params)) {
    p2$layers[[i]]$aes_params$fill <- plot_colors_light[2]
  }
  if (!is.null(p2$layers[[i]]$geom)) {
    if (inherits(p2$layers[[i]]$geom, "GeomLine")) {
      p2$layers[[i]]$aes_params$colour <- plot_colors[2]
    }
    if (inherits(p2$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      p2$layers[[i]]$aes_params$fill <- plot_colors_light[2]
    }
  }
}

p3 <- draw(best_model$gam, select = "s(time_within_day_t)",
           rug = FALSE, residuals = FALSE) +
  labs(
    x = "Time since sunrise (min)",
    y = "", # Remove y-axis label
    title = NULL # Remove title
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  custom_theme +
  theme(
    legend.position = "none",
    plot.background = element_rect(fill = "white", color = NA),
    panel.background = element_rect(fill = "white", color = NA),
    axis.text.y = element_blank(), # Remove y-axis text
    axis.ticks.y = element_blank(), # Remove y-axis ticks
    plot.caption = element_blank() # Remove basis caption
  )

```

```

)

# Add darker zero line BEFORE applying color changes
p3 <- add_zero_line(p3)

for (i in seq_along(p3$layers)) {
  # Skip the zero line (GeomHline) to preserve its neutral gray color
  if (!is.null(p3$layers[[i]]$geom) && inherits(p3$layers[[i]]$geom, "GeomHline")) {
    next
  }

  if ("colour" %in% names(p3$layers[[i]]$aes_params)) {
    p3$layers[[i]]$aes_params$colour <- plot_colors[3]
  }
  if ("fill" %in% names(p3$layers[[i]]$aes_params)) {
    p3$layers[[i]]$aes_params$fill <- plot_colors_light[3]
  }
  if (!is.null(p3$layers[[i]]$geom)) {
    if (inherits(p3$layers[[i]]$geom, "GeomLine")) {
      p3$layers[[i]]$aes_params$colour <- plot_colors[3]
    }
    if (inherits(p3$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      p3$layers[[i]]$aes_params$fill <- plot_colors_light[3]
    }
  }
}

p4 <- draw(best_model$gam, select = "s(temperature_avg)",
           rug = FALSE, residuals = FALSE) +
  labs(
    x = "Temperature (°C)",
    y = "", # Remove y-axis label
    title = NULL # Remove title
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  custom_theme +
  theme(
    legend.position = "none",
    plot.background = element_rect(fill = "white", color = NA),
    panel.background = element_rect(fill = "white", color = NA),
    axis.text.y = element_blank(), # Remove y-axis text
    axis.ticks.y = element_blank(), # Remove y-axis ticks
    plot.caption = element_blank() # Remove basis caption
  )

```

```

# Add darker zero line BEFORE applying color changes
p4 <- add_zero_line(p4)

for (i in seq_along(p4$layers)) {
  # Skip the zero line (GeomHline) to preserve its neutral gray color
  if (!is.null(p4$layers[[i]]$geom) && inherits(p4$layers[[i]]$geom, "GeomHline")) {
    next
  }

  if ("colour" %in% names(p4$layers[[i]]$aes_params)) {
    p4$layers[[i]]$aes_params$colour <- plot_colors[4]
  }
  if ("fill" %in% names(p4$layers[[i]]$aes_params)) {
    p4$layers[[i]]$aes_params$fill <- plot_colors_light[4]
  }
  if (!is.null(p4$layers[[i]]$geom)) {
    if (inherits(p4$layers[[i]]$geom, "GeomLine")) {
      p4$layers[[i]]$aes_params$colour <- plot_colors[4]
    }
    if (inherits(p4$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      p4$layers[[i]]$aes_params$fill <- plot_colors_light[4]
    }
  }
}

# Add flight threshold area (12.7-16°C) with fuzzy edges AFTER color modifications
# Create a gradient effect by layering rectangles with decreasing alpha
fade_width <- 0.5 # Width of fade zone on each side

# Core threshold area (solid)
p4 <- p4 +
  annotate("rect",
    xmin = 12.7 + fade_width, xmax = 16 - fade_width,
    ymin = -Inf, ymax = Inf,
    fill = "#ADD8E6", alpha = 0.4)

# Fade zones - multiple layers for smooth gradient
fade_steps <- 5
for (i in 1:fade_steps) {
  alpha_val <- 0.4 * (fade_steps - i + 1) / fade_steps
  fade_offset <- fade_width * i / fade_steps

  # Left fade
  p4 <- p4 +
    annotate("rect",

```

```

        xmin = 12.7 + fade_width - fade_offset,
        xmax = 12.7 + fade_width - fade_offset + fade_width/fade_steps,
        ymin = -Inf, ymax = Inf,
        fill = "#ADD8E6", alpha = alpha_val)

# Right fade
p4 <- p4 +
  annotate("rect",
    xmin = 16 - fade_width + fade_offset - fade_width/fade_steps,
    xmax = 16 - fade_width + fade_offset,
    ymin = -Inf, ymax = Inf,
    fill = "#ADD8E6", alpha = alpha_val)
}

# Display individual plots
print(p1)
print(p2)
print(p3)
print(p4)
}

```

Skipping detailed effect plots: expected smooths not present in best model

1x4 Grid Layout (Wide Aspect Ratio)

```

# Check that all plots exist before combining
if (isTRUE(has_expected) && exists("p1") && exists("p2") && exists("p3") && exists("p4")) {
  # Create 1x4 combined plot with explicit layout
  combined_1x4 <- wrap_plots(p1, p2, p3, p4, nrow = 1, ncol = 4) +
    plot_annotation(
      theme = theme(plot.title = element_text(size = 16, hjust = 0.5, face = "bold"))
    )

  print(combined_1x4)

  # Save the 1x4 layout to exports folder immediately
  ggsave(file.path(export_dir, "figures", "partial_effects_1x4_display.png"), combined_1x4,
    width = 16, height = 5, dpi = 300, bg = "white"
  )
  cat("1x4 display plot saved to:", file.path(export_dir, "figures", "partial_effects_1x4_d"), "\n")
} else {
  cat("Skipping combined 1x4 plot: effects unavailable\n")
}

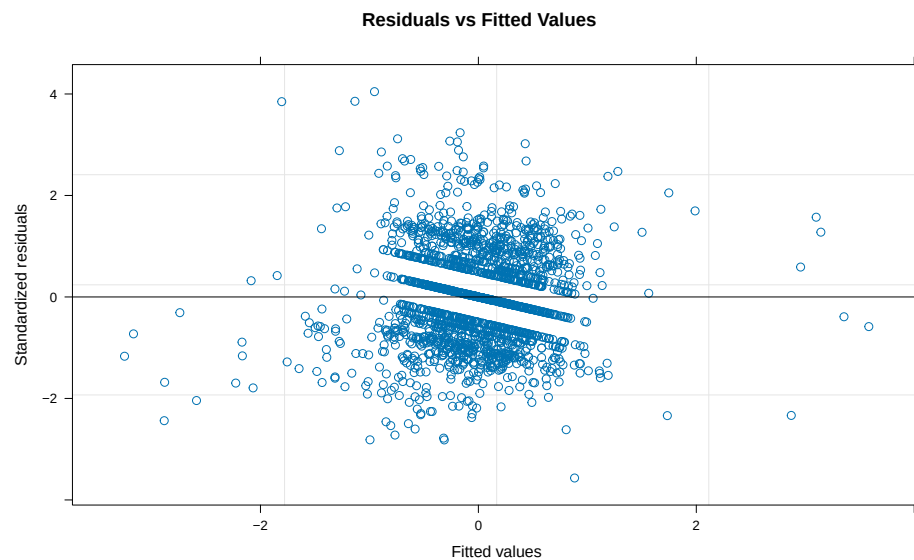
```

Skipping combined 1x4 plot: effects unavailable

Model Diagnostics

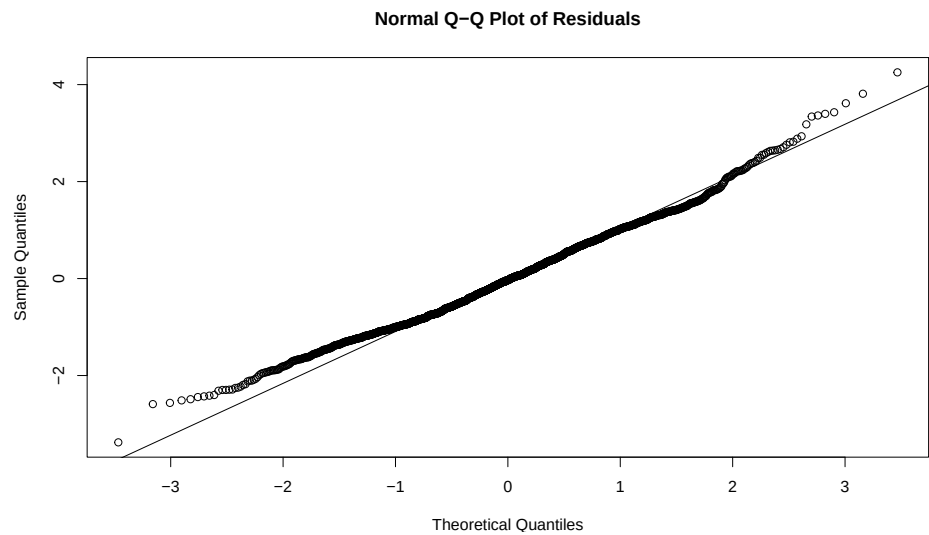
Residuals vs Fitted Values

```
plot(best_model$lme, main = "Residuals vs Fitted Values")
```



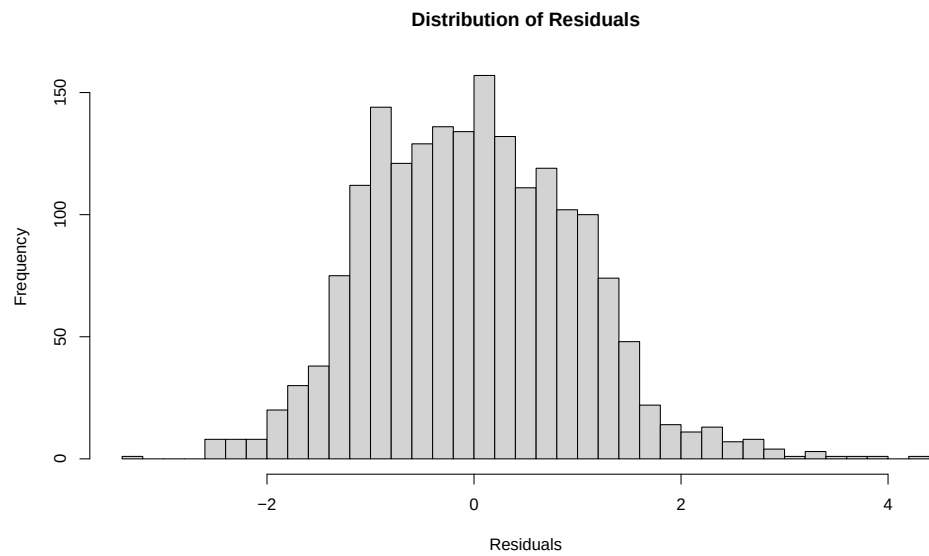
Q-Q Plot of Residuals

```
residuals_df <- data.frame(  
  fitted = fitted(best_model$lme),  
  residuals = residuals(best_model$lme, type = "normalized")  
)  
  
qqnorm(residuals_df$residuals, main = "Normal Q-Q Plot of Residuals")  
qqline(residuals_df$residuals)
```



Distribution of Residuals

```
hist(residuals_df$residuals, main = "Distribution of Residuals", xlab = "Residuals", breaks
```



Results Summary

This analysis provides robust evidence regarding wind effects on overwintering monarch butterfly movement through comprehensive model comparison across

47 candidate models. The results reveal several key findings:

Wind Effects: Wind was not selected in the best-performing model and only appeared once in the top 5 models (plotted above) with a non-significant effect ($p = 0.218$). This suggests that wind is not a primary driver of short-term monarch movement patterns at the temporal and spatial scales examined.

Primary Drivers: Temperature and diurnal patterns emerged as the strongest predictors of monarch movement. The best model revealed non-linear temperature responses with apparent thermal optima, and strong diurnal cycles consistent with monarch thermoregulatory behavior.

Model Performance: Including smooth terms substantially improved model fit (R^2 increased from 2.74% to 5.61%), highlighting the importance of capturing non-linear relationships in ecological modeling.

Sensitivity analyses focused on threshold-based wind metrics are documented separately in `analysis/monarch_threshold_analysis.qmd`. ## Publication-Ready Exports

The following section exports all figures and tables needed for the thesis results section.

```
# Set up export directory
export_dir <- here("thesis_exports")
fig_dir <- file.path(export_dir, "figures")
tab_dir <- file.path(export_dir, "tables")
thesis_results_fig_dir <- here("../", "masters-thesis", "figures", "results")
thesis_results_tab_dir <- here("../", "masters-thesis", "supplemental", "results", "thesis_ex

# Create directories if they don't exist
if (!dir.exists(export_dir)) dir.create(export_dir, recursive = TRUE)
if (!dir.exists(fig_dir)) dir.create(fig_dir, recursive = TRUE)
if (!dir.exists(tab_dir)) dir.create(tab_dir, recursive = TRUE)
if (!dir.exists(thesis_results_fig_dir)) dir.create(thesis_results_fig_dir, recursive = TRUE)
if (!dir.exists(thesis_results_tab_dir)) dir.create(thesis_results_tab_dir, recursive = TRUE)

# Load additional packages for publication-ready plots
library(ggthemes)
library(scales)

# Define publication theme
pub_theme <- theme_classic(base_size = 12) +
  theme(
    axis.text = element_text(color = "black"),
    axis.title = element_text(color = "black"),
    legend.text = element_text(color = "black"),
    legend.title = element_text(color = "black"),
```

```

    strip.text = element_text(color = "black"),
    plot.title = element_text(color = "black", hjust = 0.5),
    panel.grid.major = element_line(color = "gray90", size = 0.5),
    panel.grid.minor = element_blank(),
    legend.position = "bottom"
)

cat("Export directories created successfully\n")

```

Export directories created successfully

Wind × Sun Interaction (Binned, High-Res)

```

# Source helper for consistent binned interaction plots
source(here("analysis", "plot_binned_interaction.R"))

# Ensure export directories exist when running this chunk standalone
if (!exists("fig_dir")) {
  export_dir <- here("thesis_exports")
  fig_dir <- file.path(export_dir, "figures")
  if (!dir.exists(export_dir)) dir.create(export_dir, recursive = TRUE)
  if (!dir.exists(fig_dir)) dir.create(fig_dir, recursive = TRUE)
}
if (!exists("thesis_results_fig_dir")) {
  thesis_results_fig_dir <- here("../", "masters-thesis", "figures", "results")
}
if (!dir.exists(thesis_results_fig_dir)) {
  dir.create(thesis_results_fig_dir, recursive = TRUE)
}

# Check if the best model includes the wind × sun tensor product
has_wind_sun_ti <- tryCatch({
  any(grepl(
    "ti\\(max_gust,butterflies_direct_sun_t_lag\\)",
    rownames(summary(best_model$gam)$s.table)
  ))
}, error = function(e) FALSE)

if (isTRUE(has_wind_sun_ti)) {
  # Create the binned, high-resolution interaction surface plot
  p_inter_binned <- create_binned_interaction_plot(
    gam_model = best_model$gam,
    x_var = "max_gust",

```

```

y_var = "butterflies_direct_sun_t_lag",
data = model_data,
title = "Wind × Sun Interaction",
subtitle = paste0(
  "Binned visualization | Best model ", best_model_name,
  " | AIC = ", sprintf("%.1f", AIC(best_model$lme))
),
xlab = "Maximum wind speed (m/s)",
ylab = "Butterflies in direct sun",
n = 400,
# More categories with fine resolution near zero
limits = c(-6, 6),
nbreaks = 17,
breaks = c(-6, -5, -4, -3, -2, -1, -0.5, 0, 0.5, 1, 2, 3, 4, 5, 6),
labels = c("-6", "-5", "-4", "-3", "-2", "-1", "-0.5", "0", "+0.5", "+1", "+2", "+3", "+4", "+5", "+6"),
barheight = 34,
barwidth = 1.0,
legend_text_size = 8,
legend_key_height_cm = 0.9,
too_far = 0.04
)

# Add threshold reference line at 2 m/s
p_inter_binned <- p_inter_binned +
  geom_vline(xintercept = 2, color = "red", linetype = "dashed", size = 0.8, alpha = 0.8)

print(p_inter_binned)

# Save high-resolution copies
out_file <- file.path(fig_dir, "interaction_wind_x_sun_binned.png")
ggsave(out_file, p_inter_binned, width = 7, height = 6, dpi = 300, bg = "white")
cat("Binned interaction plot saved to:", out_file, "\n")

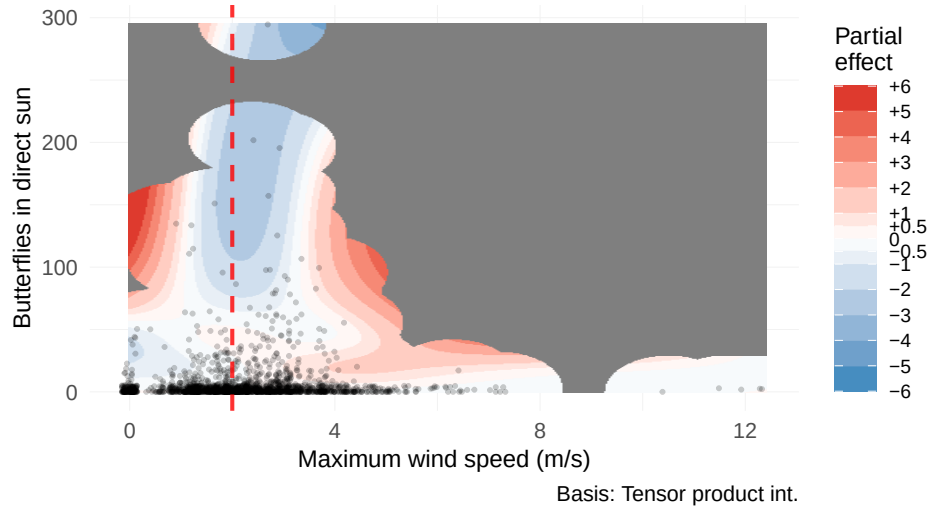
# Also save to thesis repo if path exists
if (exists("thesis_results_fig_dir") && dir.exists(thesis_results_fig_dir)) {
  out_file_thesis <- file.path(thesis_results_fig_dir, "interaction_wind_x_sun_binned.png")
  ggsave(out_file_thesis, p_inter_binned, width = 7, height = 6, dpi = 300, bg = "white")
  cat("Also saved to:", out_file_thesis, "\n")
}
} else {
  cat("Best model does not include ti(max_gust, butterflies_direct_sun_t_lag); skipping interaction plot")
}
}

```

Binned interaction plot saved to: /Users/kylenessen/Documents/Code/masters-analysis/thesis_repo

Wind <U+00D7> Sun Interaction

Binned visualization | Best model M50 | AIC = 8078.0



Also saved to: /Users/kylenessen/Documents/Code/masters-analysis/./masters-thesis/figures/1

Model Selection Table Export

```
# Function to extract max_gust p-value from a model
get_max_gust_pvalue <- function(model_name) {
  if (model_name %in% names(successful_models)) {
    model <- successful_models[[model_name]]

    # Check parametric terms first (linear effects)
    param_summary <- summary(model$gam)$p.table
    if ("max_gust" %in% rownames(param_summary)) {
      return(format.pval(param_summary["max_gust", "Pr(>|t|)"], digits = 3))
    }

    # Check smooth terms
    smooth_summary <- summary(model$gam)$s.table
    gust_smooth_rows <- grep("max_gust", rownames(smooth_summary))
    if (length(gust_smooth_rows) > 0) {
      return(format.pval(smooth_summary[gust_smooth_rows[1], "p-value"], digits = 3))
    }
  }
  return("NA")
}
```

```

# Function to create readable model terms
create_readable_terms <- function(formula_str) {
  # Remove response variable
  formula_clean <- gsub("butterfly_difference_cbrt ~ ", "", formula_str, fixed = TRUE)

  # Split into terms on literal '+'
  original_terms <- trimws(strsplit(formula_clean, "+", fixed = TRUE)[[1]])
  original_terms <- original_terms[original_terms != ""]

  # Process each term individually
  formatted_terms <- vapply(original_terms, function(term) {
    term <- trimws(term)

    # Detect smooths by prefix 's(' and tensor interactions 'ti('
    is_smooth <- startsWith(term, "s(") || startsWith(term, "ti(") || startsWith(term, "

    # Helper to humanize variable names
    humanize <- function(x) {
      dplyr::case_when(
        x == "total_butterflies_t_lag" ~ "Previous butterfly count",
        x == "max_gust" ~ "Maximum wind speed",
        x == "temperature_avg" ~ "Temperature",
        x == "butterflies_direct_sun_t_lag" ~ "Butterflies in direct sun",
        x == "time_within_day_t" ~ "Time since sunrise",
        TRUE ~ x
      )
    }

    if (startsWith(term, "s(") && endsWith(term, ")")) {
      inner <- substring(term, 3, nchar(term) - 1)
      readable_name <- humanize(inner)
      return(paste0("\\textbullet\\ ", readable_name))
    }

    if ((startsWith(term, "ti(") || startsWith(term, "te(") && endsWith(term, ")")) {
      inner <- substring(term, 4, nchar(term) - 1)
      parts <- trimws(strsplit(inner, ",", fixed = TRUE)[[1]])
      parts_human <- vapply(parts, humanize, character(1))
      inter_name <- paste0(substr(term, 1, 2), "(", paste(parts_human, collapse = ", '
      return(paste0("\\textbullet\\ ", inter_name))
    }

    # Fallback: treat as linear term
    var_name <- term

    # Replace with readable names

```

```

    readable_name <- humanize(var_name)

    # Format with bullet and linear notation if needed
    paste0("\\textbullet\\ ", readable_name, " (linear)")
  }, character(1))

  if (length(formatted_terms) == 0) {
    return("• intercept only")
  }

  # Join with LaTeX line breaks and do not escape in kable
  paste(formatted_terms, collapse = "\\ \\ ")
}

# Create comprehensive model selection table (top 5 models)
model_selection_table <- aic_results %>%
  head(5) %>%
  mutate(
    AIC_weight = round(AIC_weight, 4),
    Delta_AIC = round(Delta_AIC, 3),
    AIC = round(AIC, 3),
    max_gust_p = map_chr(Model, get_max_gust_pvalue),
    Terms = map_chr(Formula, create_readable_terms)
  ) %>%
  select(Model, Terms, AIC, Delta_AIC, AIC_weight, df, max_gust_p) %>%
  rename(
    "Model ID" = Model,
    "Model Terms" = Terms,
    "AIC" = AIC,
    "ΔAIC" = Delta_AIC,
    "AIC Weight" = AIC_weight,
    "df" = df,
    "Wind p-value" = max_gust_p
  )

# Export as CSV
write_csv(model_selection_table, file.path(tab_dir, "model_selection_table.csv"))

# Export as formatted kable for LaTeX
kable_output <- kable(model_selection_table,
  format = "latex",
  booktabs = TRUE,
  escape = FALSE,
  caption = "Top 5 candidate models ranked by AIC for monarch butterfly abundance change a
)

```

```
writelnLines(kable_output, file.path(tab_dir, "model_selection_table.tex"))
```

```
cat("Model selection table exported to:", file.path(tab_dir, "model_selection_table.csv"), "
```

Model selection table exported to: /Users/kylenessen/Documents/Code/masters-analysis/thesis

```
# Also copy to thesis repository tables directory
try({
  file.copy(file.path(tab_dir, "model_selection_table.csv"),
            file.path(thesis_results_tab_dir, "model_selection_table.csv"), overwrite = TRUE)
  file.copy(file.path(tab_dir, "model_selection_table.tex"),
            file.path(thesis_results_tab_dir, "model_selection_table.tex"), overwrite = TRUE)
})
```

```
[1] TRUE
```

Smooth Terms Summary Table Export

```
# Extract smooth terms summary from best model
smooth_summary <- summary(best_model$gam)$s.table
parametric_summary <- summary(best_model$gam)$p.table

# Create formatted smooth terms table
smooth_terms_df <- data.frame(
  Term = rownames(smooth_summary),
  EDF = round(smooth_summary[, "edf"], 3),
  Ref_df = round(smooth_summary[, "Ref.df"], 3),
  F_value = round(smooth_summary[, "F"], 3),
  p_value = format.pval(smooth_summary[, "p-value"], digits = 3)
) %>%
  mutate(
    Term = case_when(
      grepl("total_butterflies_t_lag", Term) ~ "Previous butterfly count",
      grepl("temperature_avg", Term) ~ "Temperature",
      grepl("butterflies_direct_sun_t_lag", Term) ~ "Butterflies in direct sun",
      grepl("time_within_day_t", Term) ~ "Time since sunrise",
      TRUE ~ Term
    )
  )

# Export smooth terms table
write_csv(smooth_terms_df, file.path(tab_dir, "smooth_terms_summary.csv"))
```

```
# Export as LaTeX
smooth_kable <- kable(smooth_terms_df,
  format = "latex",
  booktabs = TRUE,
  col.names = c("Smooth Term", "EDF", "Ref. df", "F", "p-value"),
  caption = "Summary of smooth terms in the best-fit GAM model"
)

writeLines(smooth_kable, file.path(tab_dir, "smooth_terms_summary.tex"))

cat("Smooth terms summary exported to:", file.path(tab_dir, "smooth_terms_summary.csv"), "\n")
```

Smooth terms summary exported to: /Users/kylenessen/Documents/Code/masters-analysis/thesis_e

```
# Also copy to thesis repository tables directory
try({
  file.copy(file.path(tab_dir, "smooth_terms_summary.csv"),
    file.path(thesis_results_tab_dir, "smooth_terms_summary.csv"), overwrite = TRUE)
  file.copy(file.path(tab_dir, "smooth_terms_summary.tex"),
    file.path(thesis_results_tab_dir, "smooth_terms_summary.tex"), overwrite = TRUE)
})
```

```
[1] TRUE
```

Publication-Ready Partial Effect Plots (Updated)

```
# Skip if expected smooths are missing in best model
if (!isTRUE(has_expected)) {
  cat("Skipping partial effect plot exports: expected smooths not present in best model\n")
} else {
  # Define publication theme with grid lines and darker zero line
  pub_theme_grid <- theme_classic(base_size = 12) +
    theme(
      axis.text = element_text(color = "black", size = 11),
      axis.title = element_text(color = "black", size = 12, face = "bold"),
      legend.text = element_text(color = "black"),
      legend.title = element_text(color = "black"),
      strip.text = element_text(color = "black"),
      plot.title = element_text(color = "black", hjust = 0.5, size = 14, face = "bold"),
      panel.grid.major = element_line(color = "gray90", size = 0.5),
      panel.grid.minor = element_line(color = "gray95", size = 0.3),
      panel.border = element_rect(color = "black", fill = NA, size = 0.5),
```

```

        legend.position = "none",
        plot.margin = margin(10, 10, 10, 10)
    )

# Use the same y-axis limits and colors from the main plots
# Export individual plots with cleaned up labeling (matching your rearranged order)
# Order: Butterflies in Direct Sun, Previous Butterfly Count, Time Since Sunrise, Temperature

pe1 <- draw(best_model$gam, select = "s(butterflies_direct_sun_t_lag)",
            rug = FALSE, residuals = FALSE) +
  labs(
    title = "Butterflies in Direct Sun",
    x = "Butterflies in direct sun",
    y = "Partial effect"
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  pub_theme_grid +
  theme(plot.caption = element_blank()) # Remove Basis: TPRS

# Add darker zero line
pe1 <- add_zero_line(pe1)

# Override colors using comprehensive layer modification (skip zero line)
for (i in seq_along(pe1$layers)) {
  # Skip the zero line (GeomHline) to preserve its neutral gray color
  if (!is.null(pe1$layers[[i]]$geom) && inherits(pe1$layers[[i]]$geom, "GeomHline")) {
    next
  }

  if ("colour" %in% names(pe1$layers[[i]]$aes_params)) {
    pe1$layers[[i]]$aes_params$colour <- plot_colors[1]
  }
  if ("fill" %in% names(pe1$layers[[i]]$aes_params)) {
    pe1$layers[[i]]$aes_params$fill <- plot_colors_light[1]
  }
  if (!is.null(pe1$layers[[i]]$geom)) {
    if (inherits(pe1$layers[[i]]$geom, "GeomLine")) {
      pe1$layers[[i]]$aes_params$colour <- plot_colors[1]
    }
    if (inherits(pe1$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      pe1$layers[[i]]$aes_params$fill <- plot_colors_light[1]
    }
  }
}
}

```

```

pe2 <- draw(best_model$gam, select = "s(total_butterflies_t_lag)",
            rug = FALSE, residuals = FALSE) +
  labs(
    title = "Previous Butterfly Count",
    x = "Previous butterfly count",
    y = "" # Remove y-axis label for cleaner 1x4 layout
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  pub_theme_grid +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    plot.caption = element_blank() # Remove Basis: TPRS
  )

# Add darker zero line
pe2 <- add_zero_line(pe2)

for (i in seq_along(pe2$layers)) {
  # Skip the zero line (GeomHline) to preserve its neutral gray color
  if (!is.null(pe2$layers[[i]]$geom) && inherits(pe2$layers[[i]]$geom, "GeomHline")) {
    next
  }

  if ("colour" %in% names(pe2$layers[[i]]$aes_params)) {
    pe2$layers[[i]]$aes_params$colour <- plot_colors[2]
  }
  if ("fill" %in% names(pe2$layers[[i]]$aes_params)) {
    pe2$layers[[i]]$aes_params$fill <- plot_colors_light[2]
  }
  if (!is.null(pe2$layers[[i]]$geom)) {
    if (inherits(pe2$layers[[i]]$geom, "GeomLine")) {
      pe2$layers[[i]]$aes_params$colour <- plot_colors[2]
    }
    if (inherits(pe2$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      pe2$layers[[i]]$aes_params$fill <- plot_colors_light[2]
    }
  }
}

pe3 <- draw(best_model$gam, select = "s(time_within_day_t)",
            rug = FALSE, residuals = FALSE) +
  labs(
    title = "Time Since Sunrise",
    x = "Time since sunrise (minutes)",

```

```

    y = "" # Remove y-axis label for cleaner 1x4 layout
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  pub_theme_grid +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    plot.caption = element_blank() # Remove Basis: TPRS
  )

# Add darker zero line
pe3 <- add_zero_line(pe3)

for (i in seq_along(pe3$layers)) {
  # Skip the zero line (GeomHline) to preserve its neutral gray color
  if (!is.null(pe3$layers[[i]]$geom) && inherits(pe3$layers[[i]]$geom, "GeomHline")) {
    next
  }

  if ("colour" %in% names(pe3$layers[[i]]$aes_params)) {
    pe3$layers[[i]]$aes_params$colour <- plot_colors[3]
  }
  if ("fill" %in% names(pe3$layers[[i]]$aes_params)) {
    pe3$layers[[i]]$aes_params$fill <- plot_colors_light[3]
  }
  if (!is.null(pe3$layers[[i]]$geom)) {
    if (inherits(pe3$layers[[i]]$geom, "GeomLine")) {
      pe3$layers[[i]]$aes_params$colour <- plot_colors[3]
    }
    if (inherits(pe3$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      pe3$layers[[i]]$aes_params$fill <- plot_colors_light[3]
    }
  }
}

pe4 <- draw(best_model$gam, select = "s(temperature_avg)",
            rug = FALSE, residuals = FALSE) +
  labs(
    x = "Temperature (°C)",
    y = "", # Remove y-axis label for cleaner 1x4 layout
    title = NULL # Remove title
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  pub_theme_grid +
  theme(

```

```

    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    plot.caption = element_blank() # Remove Basis: TPRS
  )

# Add darker zero line BEFORE applying color changes
pe4 <- add_zero_line(pe4)

for (i in seq_along(pe4$layers)) {
  # Skip the zero line (GeomHline) to preserve its neutral gray color
  if (!is.null(pe4$layers[[i]]$geom) && inherits(pe4$layers[[i]]$geom, "GeomHline")) {
    next
  }

  if ("colour" %in% names(pe4$layers[[i]]$aes_params)) {
    pe4$layers[[i]]$aes_params$colour <- plot_colors[4]
  }
  if ("fill" %in% names(pe4$layers[[i]]$aes_params)) {
    pe4$layers[[i]]$aes_params$fill <- plot_colors_light[4]
  }
  if (!is.null(pe4$layers[[i]]$geom)) {
    if (inherits(pe4$layers[[i]]$geom, "GeomLine")) {
      pe4$layers[[i]]$aes_params$colour <- plot_colors[4]
    }
    if (inherits(pe4$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      pe4$layers[[i]]$aes_params$fill <- plot_colors_light[4]
    }
  }
}

# Create 2x2-specific plots with proper y-axis sharing
pe1_2x2 <- pe1 + labs(title = NULL) # Top-left: remove title, keep y-axis label

pe2_2x2 <- pe2 + labs(title = NULL) # Top-right: remove title, no y-axis

pe3_2x2 <- draw(best_model$gam, select = "s(time_within_day_t)",
  rug = FALSE, residuals = FALSE) +
  labs(
    title = NULL, # No title
    x = "Time since sunrise (minutes)",
    y = "Partial effect" # Bottom-left: keep y-axis label
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  pub_theme_grid +
  theme(plot.caption = element_blank())

```

```

pe3_2x2 <- add_zero_line(pe3_2x2)

# Apply colors to pe3_2x2
for (i in seq_along(pe3_2x2$layers)) {
  if (!is.null(pe3_2x2$layers[[i]]$geom) && inherits(pe3_2x2$layers[[i]]$geom, "GeomHline")) {
    next
  }
  if ("colour" %in% names(pe3_2x2$layers[[i]]$aes_params)) {
    pe3_2x2$layers[[i]]$aes_params$colour <- plot_colors[3]
  }
  if ("fill" %in% names(pe3_2x2$layers[[i]]$aes_params)) {
    pe3_2x2$layers[[i]]$aes_params$fill <- plot_colors_light[3]
  }
  if (!is.null(pe3_2x2$layers[[i]]$geom)) {
    if (inherits(pe3_2x2$layers[[i]]$geom, "GeomLine")) {
      pe3_2x2$layers[[i]]$aes_params$colour <- plot_colors[3]
    }
    if (inherits(pe3_2x2$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      pe3_2x2$layers[[i]]$aes_params$fill <- plot_colors_light[3]
    }
  }
}

pe4_2x2 <- draw(best_model$gam, select = "s(temperature_avg)",
  rug = FALSE, residuals = FALSE) +
  labs(
    title = NULL, # No title
    x = "Temperature (°C)",
    y = "" # Bottom-right: no y-axis label
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  pub_theme_grid +
  theme(
    axis.text.y = element_blank(),
    axis.ticks.y = element_blank(),
    plot.caption = element_blank()
  )

pe4_2x2 <- add_zero_line(pe4_2x2)

# Apply colors to pe4_2x2
for (i in seq_along(pe4_2x2$layers)) {
  if (!is.null(pe4_2x2$layers[[i]]$geom) && inherits(pe4_2x2$layers[[i]]$geom, "GeomHline")) {
    next
  }
}

```

```

    if ("colour" %in% names(pe4_2x2$layers[[i]]$aes_params)) {
      pe4_2x2$layers[[i]]$aes_params$colour <- plot_colors[4]
    }
    if ("fill" %in% names(pe4_2x2$layers[[i]]$aes_params)) {
      pe4_2x2$layers[[i]]$aes_params$fill <- plot_colors_light[4]
    }
    if (!is.null(pe4_2x2$layers[[i]]$geom)) {
      if (inherits(pe4_2x2$layers[[i]]$geom, "GeomLine")) {
        pe4_2x2$layers[[i]]$aes_params$colour <- plot_colors[4]
      }
      if (inherits(pe4_2x2$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
        pe4_2x2$layers[[i]]$aes_params$fill <- plot_colors_light[4]
      }
    }
  }
}

# Add flight threshold area (12.7-16°C) with fuzzy edges to pe4_2x2 AFTER color modification
# Create a gradient effect by layering rectangles with decreasing alpha
fade_width <- 0.5 # Width of fade zone on each side

# Core threshold area (solid)
pe4_2x2 <- pe4_2x2 +
  annotate("rect",
    xmin = 12.7 + fade_width, xmax = 16 - fade_width,
    ymin = -Inf, ymax = Inf,
    fill = "#ADD8E6", alpha = 0.4)

# Fade zones - multiple layers for smooth gradient
fade_steps <- 5
for (i in 1:fade_steps) {
  alpha_val <- 0.4 * (fade_steps - i + 1) / fade_steps
  fade_offset <- fade_width * i / fade_steps

  # Left fade
  pe4_2x2 <- pe4_2x2 +
    annotate("rect",
      xmin = 12.7 + fade_width - fade_offset,
      xmax = 12.7 + fade_width - fade_offset + fade_width/fade_steps,
      ymin = -Inf, ymax = Inf,
      fill = "#ADD8E6", alpha = alpha_val)

  # Right fade
  pe4_2x2 <- pe4_2x2 +
    annotate("rect",
      xmin = 16 - fade_width + fade_offset - fade_width/fade_steps,

```

```

        xmax = 16 - fade_width + fade_offset,
        ymin = -Inf, ymax = Inf,
        fill = "#ADD8E6", alpha = alpha_val)
}

# Export individual plots
ggsave(file.path(fig_dir, "effect_previous_butterfly_count_final.png"), pe1,
        width = 8, height = 6, dpi = 300, bg = "white"
)
ggsave(file.path(fig_dir, "effect_temperature_final.png"), pe2,
        width = 8, height = 6, dpi = 300, bg = "white"
)
ggsave(file.path(fig_dir, "effect_butterflies_direct_sun_final.png"), pe3,
        width = 8, height = 6, dpi = 300, bg = "white"
)
ggsave(file.path(fig_dir, "diurnal_patterns_final.png"), pe4,
        width = 8, height = 6, dpi = 300, bg = "white"
)

# Create and export 2x2 combined figure with proper y-axis sharing
tryCatch({
  combined_pub_2x2 <- wrap_plots(pe1_2x2, pe2_2x2, pe3_2x2, pe4_2x2, nrow = 2, ncol = 2)

  ggsave(file.path(fig_dir, "combined_partial_effects_2x2_final.png"), combined_pub_2x2,
          width = 12, height = 10, dpi = 300, bg = "white"
  )
  cat("2x2 combined plot exported successfully\n")
}, error = function(e) {
  cat("Error creating 2x2 combined plot:", e$message, "\n")
})

# Create and export 1x4 combined figure (wide aspect ratio)
tryCatch({
  combined_pub_1x4 <- wrap_plots(pe1, pe2, pe3, pe4, nrow = 1, ncol = 4) # No title

  ggsave(file.path(fig_dir, "combined_partial_effects_1x4_final.png"), combined_pub_1x4,
          width = 16, height = 5, dpi = 300, bg = "white"
  )
  cat("1x4 combined plot exported successfully\n")
}, error = function(e) {
  cat("Error creating 1x4 combined plot:", e$message, "\n")
})

# Mirror key figures to thesis repo figures directory
try({

```

```

if (!dir.exists(thesis_results_fig_dir)) dir.create(thesis_results_fig_dir, recursive = TRUE)
fig_files <- c(
  "effect_previous_butterfly_count_final.png",
  "effect_temperature_final.png",
  "effect_butterflies_direct_sun_final.png",
  "diurnal_patterns_final.png",
  "combined_partial_effects_2x2_final.png",
  "combined_partial_effects_1x4_final.png"
)
for (f in fig_files) {
  src <- file.path(fig_dir, f)
  if (file.exists(src)) file.copy(src, file.path(thesis_results_fig_dir, f), overwrite = TRUE)
}
})

# Export standalone partial effect for maximum wind speed from model M24
if ("M24" %in% names(successful_models)) {
  model_m24 <- successful_models[["M24"]]
  wind_color <- "#4C8BB9"
  wind_color_light <- lighten_color(wind_color)

  pe_wind_m24 <- draw(model_m24$gam, select = "s(max_gust)", rug = FALSE, residuals = FALSE)
  labs(
    title = NULL,
    x = "Maximum wind speed (m/s)",
    y = "Partial effect"
  ) +
  scale_y_continuous(limits = y_limits, breaks = y_breaks) +
  pub_theme_grid +
  theme(plot.caption = element_blank())

  pe_wind_m24 <- add_zero_line(pe_wind_m24)

  for (i in seq_along(pe_wind_m24$layers)) {
    if (!is.null(pe_wind_m24$layers[[i]]$geom) && inherits(pe_wind_m24$layers[[i]]$geom, "GeomLine")) {
      next
    }

    if ("colour" %in% names(pe_wind_m24$layers[[i]]$aes_params)) {
      pe_wind_m24$layers[[i]]$aes_params$colour <- wind_color
    }
    if ("fill" %in% names(pe_wind_m24$layers[[i]]$aes_params)) {
      pe_wind_m24$layers[[i]]$aes_params$fill <- wind_color_light
    }
    if (!is.null(pe_wind_m24$layers[[i]]$geom)) {

```

```

    if (inherits(pe_wind_m24$layers[[i]]$geom, "GeomLine")) {
      pe_wind_m24$layers[[i]]$aes_params$colour <- wind_color
    }
    if (inherits(pe_wind_m24$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) {
      pe_wind_m24$layers[[i]]$aes_params$fill <- wind_color_light
    }
  }
}

pe_wind_m24 <- pe_wind_m24 +
  geom_vline(xintercept = 2, color = "red", linetype = "dashed", size = 0.8)

ggsave(file.path(fig_dir, "m24_wind_partial_effect.png"), pe_wind_m24,
  width = 6, height = 5, dpi = 300, bg = "white"
)

thesis_wind_path <- file.path(thesis_results_fig_dir, "m24_wind_partial_effect.png")
ggsave(thesis_wind_path, pe_wind_m24,
  width = 6, height = 5, dpi = 300, bg = "white"
)

cat("M24 wind partial effect exported successfully\n")
cat("- Saved to thesis_exports/figures/m24_wind_partial_effect.png\n")
cat("- Saved to ../masters-thesis/figures/results/m24_wind_partial_effect.png\n")
} else {
  cat("Model M24 unavailable; skipping wind partial effect export\n")
}

cat("Updated publication-ready plots exported to:", fig_dir, "\n")
cat("Files created:\n")
cat("Individual plots:\n")
cat("- effect_previous_butterfly_count_final.png\n")
cat("- effect_temperature_final.png\n")
cat("- effect_butterflies_direct_sun_final.png\n")
cat("- diurnal_patterns_final.png\n")
cat("- m24_wind_partial_effect.png (also saved to ../masters-thesis/figures/results/)\n")
cat("Combined layouts:\n")
cat("- combined_partial_effects_2x2_final.png\n")
cat("- combined_partial_effects_1x4_final.png (16x5, wide aspect ratio)\n")
cat("\nAll plots feature:\n")
cat("- Custom hex color palette\n")
cat("- 10% lighter confidence intervals\n")
cat("- Darker zero reference line (gray50)\n")
  cat("- Clean 1x4 layout with single y-axis label\n")
}

```

Skipping partial effect plot exports: expected smooths not present in best model

Publication-Ready Partial Effects (Smart 1x3 without Sun)

```
# Build partial effects for the smooths present in best model (exclude direct sun)
library(gratia)
library(patchwork)

# Identify smooth labels present
available_smooths <- rownames(summary(best_model$gam)$s.table)
lab_prev <- available_smooths[grepl("^s\\(total_butterflies_t_lag\\)", available_smooths)]
lab_temp <- available_smooths[grepl("^s\\(temperature_avg\\)", available_smooths)]
lab_time <- available_smooths[grepl("^s\\(time_within_day_t\\)", available_smooths)]

# Colors
col_prev <- "#9673c5" # purple
col_time <- "#79a44c" # green
col_temp <- "#b86e7e" # mauve/pink
lighten <- function(hex, amount = 0.1) {
  rgb_vals <- col2rgb(hex)
  lightened <- rgb_vals + (255 - rgb_vals) * amount
  rgb(lightened[1]/255, lightened[2]/255, lightened[3]/255)
}

# Helper to recolor gratia::draw plot
recolor <- function(p, col_line, col_fill) {
  # Add the neutral zero line first
  p$layers <- c(list(geom_hline(yintercept = 0, color = "gray70", size = 0.8)), p$layers)
  for (i in seq_along(p$layers)) {
    if (!is.null(p$layers[[i]]$geom) && inherits(p$layers[[i]]$geom, "GeomHline")) next
    if ("colour" %in% names(p$layers[[i]]$aes_params)) p$layers[[i]]$aes_params$colour <- col_line
    if ("fill" %in% names(p$layers[[i]]$aes_params)) p$layers[[i]]$aes_params$fill <- col_fill
    if (!is.null(p$layers[[i]]$geom)) {
      if (inherits(p$layers[[i]]$geom, "GeomLine")) p$layers[[i]]$aes_params$colour <- col_line
      if (inherits(p$layers[[i]]$geom, c("GeomRibbon", "GeomPolygon"))) p$layers[[i]]$aes_params$fill <- col_fill
    }
  }
  p
}

plots <- list()
ymins <- c(); ymaxs <- c()

if (length(lab_prev) == 1) {
```

```

p_prev <- draw(best_model$gam, select = lab_prev, rug = FALSE, residuals = FALSE) +
  labs(title = "Previous Butterfly Count", x = "Previous butterfly count", y = "Partial effect") +
  theme_classic(base_size = 12) + theme(plot.caption = element_blank())
plots$prev <- recolor(p_prev, col_prev, lighten(col_prev))
rr <- ggplot_build(plots$prev)$layout$panel_scales_y[[1]]$range$range
ymins <- c(ymins, rr[1]); ymaxs <- c(ymaxs, rr[2])
}

if (length(lab_time) == 1) {
  p_time <- draw(best_model$gam, select = lab_time, rug = FALSE, residuals = FALSE) +
    labs(title = "Time Since Sunrise", x = "Time since sunrise (minutes)", y = "Partial effect") +
    theme_classic(base_size = 12) + theme(plot.caption = element_blank())
plots$time <- recolor(p_time, col_time, lighten(col_time))
rr <- ggplot_build(plots$time)$layout$panel_scales_y[[1]]$range$range
ymins <- c(ymins, rr[1]); ymaxs <- c(ymaxs, rr[2])
}

if (length(lab_temp) == 1) {
  p_temp <- draw(best_model$gam, select = lab_temp, rug = FALSE, residuals = FALSE) +
    labs(title = "Temperature", x = "Temperature (°C)", y = "Partial effect") +
    theme_classic(base_size = 12) + theme(plot.caption = element_blank())
plots$temp <- recolor(p_temp, col_temp, lighten(col_temp))
rr <- ggplot_build(plots$temp)$layout$panel_scales_y[[1]]$range$range
ymins <- c(ymins, rr[1]); ymaxs <- c(ymaxs, rr[2])
}

if (length(plots) > 0) {
  y_min <- floor(min(ymins)); y_max <- 1.5
  fix_y <- function(p) p + scale_y_continuous(limits = c(y_min, y_max), breaks = seq(y_min,
  plots <- lapply(plots, fix_y)

  # Compose 1x3 layout in preferred order: previous, time, temperature
  combo <- wrap_plots(plots$prev, plots$time, plots$temp, nrow = 1, ncol = length(plots))
  out1x3 <- file.path(fig_dir, "combined_partial_effects_1x3_final.png")
  ggsave(out1x3, combo, width = max(12, 4 * length(plots)), height = 5, dpi = 300, bg = "white")
  cat("1x3 partial effects exported to:", out1x3, "\n")

  # Mirror to thesis repo
  try({
    if (!dir.exists(thesis_results_fig_dir)) dir.create(thesis_results_fig_dir, recursive = TRUE)
    file.copy(out1x3, file.path(thesis_results_fig_dir, basename(out1x3)), overwrite = TRUE)
  })
} else {
  cat("No partial effects to export for current best model.\n")
}

```

1x3 partial effects exported to: /Users/kylenessen/Documents/Code/masters-analysis/thesis_results_fig_dir/combined_partial_effects_1x3_final.png

```
[1] TRUE
```

Model Diagnostics Plots Export

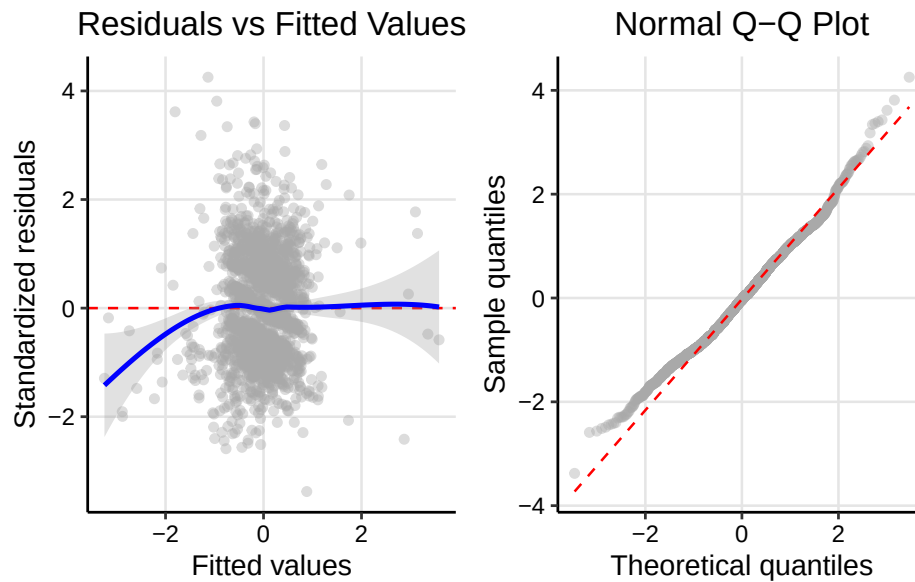
```
# Create publication-ready residuals vs fitted plot
residuals_df <- data.frame(
  fitted = fitted(best_model$lme),
  residuals = residuals(best_model$lme, type = "normalized")
)

p_resid <- ggplot(residuals_df, aes(x = fitted, y = residuals)) +
  geom_point(alpha = 0.4, color = "darkgray") +
  geom_hline(yintercept = 0, linetype = "dashed", color = "red") +
  geom_smooth(method = "loess", se = TRUE, color = "blue", alpha = 0.3) +
  labs(
    title = "Residuals vs Fitted Values",
    x = "Fitted values",
    y = "Standardized residuals"
  ) +
  pub_theme

# Create publication-ready Q-Q plot
p_qq <- ggplot(residuals_df, aes(sample = residuals)) +
  stat_qq(alpha = 0.4, color = "darkgray") +
  stat_qq_line(color = "red", linetype = "dashed") +
  labs(
    title = "Normal Q-Q Plot",
    x = "Theoretical quantiles",
    y = "Sample quantiles"
  ) +
  pub_theme

# Export diagnostic plots
ggsave(file.path(fig_dir, "residuals_vs_fitted.png"), p_resid,
  width = 8, height = 6, dpi = 300, bg = "white"
)
ggsave(file.path(fig_dir, "qq_plot.png"), p_qq,
  width = 8, height = 6, dpi = 300, bg = "white"
)

# Combined diagnostics plot
combined_diagnostics <- gridExtra::grid.arrange(p_resid, p_qq, ncol = 2)
```



```
ggsave(file.path(fig_dir, "combined_diagnostics.png"), combined_diagnostics,
        width = 12, height = 6, dpi = 300, bg = "white"
)

cat("Diagnostic plots exported to:", fig_dir, "\n")
```

Diagnostic plots exported to: /Users/kylenessen/Documents/Code/masters-analysis/thesis_exports

```
# Mirror diagnostics to thesis repo
try({
  if (!dir.exists(thesis_results_fig_dir)) dir.create(thesis_results_fig_dir, recursive = TRUE)
  diag_files <- c("residuals_vs_fitted.png", "qq_plot.png", "combined_diagnostics.png")
  for (f in diag_files) {
    src <- file.path(fig_dir, f)
    if (file.exists(src)) file.copy(src, file.path(thesis_results_fig_dir, f), overwrite = TRUE)
  }
})
```

Wind Hypothesis Evaluation Plot

```
# Create publication-ready wind scatter plot
wind_cor <- cor(model_data$max_gust, model_data$butterfly_difference_cbirt, use = "complete.obs")

p_wind <- ggplot(model_data, aes(x = max_gust, y = butterfly_difference_cbirt)) +
```

```

geom_point(alpha = 0.4, color = "darkgray") +
geom_smooth(method = "lm", se = TRUE, color = "steelblue", alpha = 0.3) +
geom_hline(yintercept = 0, linetype = "dashed", color = "red", alpha = 0.7) +
geom_vline(xintercept = 2, linetype = "dashed", color = "red", alpha = 0.7) +
labs(
  title = "Maximum Wind Speed vs Butterfly Abundance Change",
  subtitle = paste0("Correlation: r = ", round(wind_cor, 3)),
  x = "Maximum wind speed (m/s)",
  y = "Butterfly abundance change (cube root transformed)"
) +
pub_theme

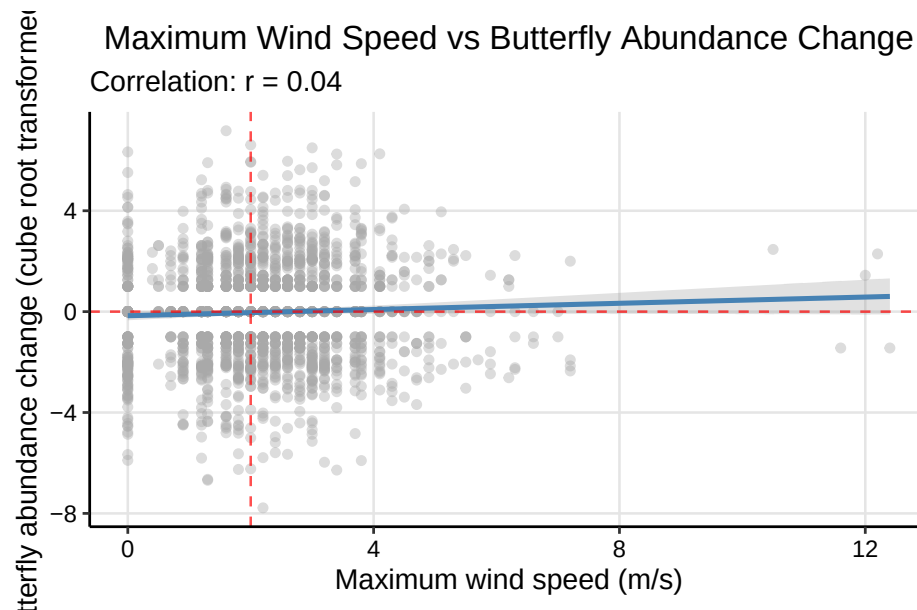
ggsave(file.path(fig_dir, "wind_hypothesis_scatter.png"), p_wind,
  width = 8, height = 6, dpi = 300, bg = "white"
)

cat("Wind hypothesis scatter plot exported to:", fig_dir, "\n")

```

Wind hypothesis scatter plot exported to: /Users/kylenessen/Documents/Code/masters-analysis/

p_wind



```

# Mirror to thesis repo
try({

```

```

if (!dir.exists(thesis_results_fig_dir)) dir.create(thesis_results_fig_dir, recursive = TRUE)
src <- file.path(fig_dir, "wind_hypothesis_scatter.png")
if (file.exists(src)) file.copy(src, file.path(thesis_results_fig_dir, "wind_hypothesis_scatter.png"))
})

```

```
[1] TRUE
```

Data Summary Export

```

# Create data summary statistics
data_summary <- model_data %>%
  summarise(
    N_observations = n(),
    N_deployments = n_distinct(deployment_id),
    N_deployment_days = n_distinct(deployment_day),
    Mean_butterflies = round(mean(total_butterflies_t_lag, na.rm = TRUE), 1),
    SD_butterflies = round(sd(total_butterflies_t_lag, na.rm = TRUE), 1),
    Mean_temp = round(mean(temperature_avg, na.rm = TRUE), 1),
    SD_temp = round(sd(temperature_avg, na.rm = TRUE), 1),
    Mean_wind = round(mean(max_gust, na.rm = TRUE), 1),
    SD_wind = round(sd(max_gust, na.rm = TRUE), 1)
  ) %>%
  pivot_longer(everything(), names_to = "Statistic", values_to = "Value") %>%
  mutate(
    Statistic = case_when(
      Statistic == "N_observations" ~ "Total observations",
      Statistic == "N_deployments" ~ "Unique deployments",
      Statistic == "N_deployment_days" ~ "Deployment-day combinations",
      Statistic == "Mean_butterflies" ~ "Mean butterfly count",
      Statistic == "SD_butterflies" ~ "SD butterfly count",
      Statistic == "Mean_temp" ~ "Mean temperature (°C)",
      Statistic == "SD_temp" ~ "SD temperature (°C)",
      Statistic == "Mean_wind" ~ "Mean wind gust (m/s)",
      Statistic == "SD_wind" ~ "SD wind gust (m/s)",
      TRUE ~ Statistic
    )
  )

write_csv(data_summary, file.path(tab_dir, "data_summary.csv"))

# Export as LaTeX table
summary_kable <- kable(data_summary,
  format = "latex",

```

```

    booktabs = TRUE,
    col.names = c("Statistic", "Value"),
    caption = "Summary statistics for the monarch butterfly analysis dataset"
  )

writeLines(summary_kable, file.path(tab_dir, "data_summary.tex"))

cat("Data summary exported to:", file.path(tab_dir, "data_summary.csv"), "\n")

```

Data summary exported to: /Users/kyleneessen/Documents/Code/masters-analysis/thesis_exports/t

Export Summary Report

```

# Create a summary file with all export information
export_summary <- paste0(
  "# Monarch GAM Analysis - Thesis Exports\n\n",
  "Export completed on: ", Sys.time(), "\n\n",
  "## Files Created:\n\n",
  "### Tables:\n",
  "- model_selection_table.csv/tex: Top 10 models by AIC\n",
  "- smooth_terms_summary.csv/tex: Summary of smooth terms in best model\n",
  "- data_summary.csv/tex: Dataset summary statistics\n\n",
  "### Figures:\n",
  "- effect_previous_butterfly_count.png: Partial effect of previous butterfly count\n",
  "- effect_temperature.png: Partial effect of temperature\n",
  "- effect_butterflies_direct_sun.png: Partial effect of butterflies in direct sun\n",
  "- diurnal_patterns.png: Diurnal patterns\n",
  "- combined_partial_effects.png: All partial effects in one figure\n",
  "- residuals_vs_fitted.png: Model diagnostic plot\n",
  "- qq_plot.png: Normal Q-Q plot of residuals\n",
  "- combined_diagnostics.png: Both diagnostic plots combined\n",
  "- wind_hypothesis_scatter.png: Wind speed vs butterfly change scatter plot\n\n",
  "## Key Results for Thesis:\n",
  "- Best model: ", best_model_name, "\n",
  "- Best model AIC: ", round(min(aic_results$AIC), 3), "\n",
  "- Model R2: ", round(summary(best_model$gam)$r.sq, 4), "\n",
  "- Wind hypothesis: NOT SUPPORTED (wind not in best model)\n\n",
  "All files are publication-ready with 300 DPI resolution and clean formatting."
)

writeLines(export_summary, file.path(export_dir, "export_summary.txt"))

## Mirror exports to thesis repository (figures + tables)

```

```

try({
  # Ensure dirs exist
  if (!dir.exists(thesis_results_fig_dir)) dir.create(thesis_results_fig_dir, recursive = T)
  if (!dir.exists(thesis_results_tab_dir)) dir.create(thesis_results_tab_dir, recursive = T)

  # Figures to mirror
  fig_list <- c(
    "combined_partial_effects_2x2_final.png",
    "combined_partial_effects_1x4_final.png",
    "effect_previous_butterfly_count_final.png",
    "effect_temperature_final.png",
    "effect_butterflies_direct_sun_final.png",
    "diurnal_patterns_final.png",
    "combined_diagnostics.png",
    "wind_hypothesis_scatter.png",
    "m24_wind_partial_effect.png",
    "interaction_wind_x_sun_binned.png"
  )
  for (f in fig_list) {
    src <- file.path(fig_dir, f)
    if (file.exists(src)) file.copy(src, file.path(thesis_results_fig_dir, f), overwrite = T)
  }

  # Tables to mirror
  tab_list <- c(
    "model_selection_table.csv", "model_selection_table.tex",
    "smooth_terms_summary.csv", "smooth_terms_summary.tex",
    "data_summary.csv", "data_summary.tex"
  )
  for (t in tab_list) {
    src <- file.path(tab_dir, t)
    if (file.exists(src)) file.copy(src, file.path(thesis_results_tab_dir, t), overwrite = T)
  }
})

cat("\n=== EXPORT COMPLETE ===\n")

```

=== EXPORT COMPLETE ===

```
cat("All publication-ready files exported to:", export_dir, "\n")
```

All publication-ready files exported to: /Users/kyleneessen/Documents/Code/masters-analysis/t

```
cat("Check export_summary.txt for complete file listing\n")
```

Check export_summary.txt for complete file listing